UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

**FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS**



INFORME TÉCNICO DE INVESTIGACIÓN FORMATIVA

"RUTAS MÍNIMAS CON DISTINTAS COLAS DE PRIORIDAD EN REDES VIALES URBANAS DE LA CIUDAD DEL CUSCO"

Asignatura: ALGORITMOS AVANZADOS (Semestre 2026-2)

Docente Responsable: Ing. Héctor Eduardo Ugarte Rojas

Identificador: GRUPO 5

Presentado por:

- Choquenaira Quispe, Noe Franklin 133962
- Porroa Sivana, Yeni Ruth 120893
- Quispe Rimachi, Romario 164257
- Yaranga Achahui, Aldo 103179

Cusco – Perú – 2026

1. Resumen

Este proyecto aborda la determinación eficiente de rutas mínimas en redes viales urbanas mediante el Algoritmo de Dijkstra instrumentado con tres estructuras de colas de prioridad: Min-Heap Binario Indexado, 4-ary Heap ($d = 4$) optimizado para memoria caché y Fibonacci Heap amortizado. La investigación integra un caso de estudio aplicado sobre la red vial real del Cusco extraída de OpenStreetMap (891 vértices y 2,139 aristas dirigidas), evaluada bajo una función de costo generalizado que incorpora la topografía andina y pendientes longitudinales severas, junto con una experimentación controlada sobre instancias sintéticas reproducibles en perfiles disperso ($|E| = 4|V|$) y denso ($|E| = 0.08\,|V|(|V|-1)$) en diez escalas de tamaño. El protocolo de medición utiliza reloj de nanosegundos (`perf_counter_ns`), aislamiento estricto del recolector de basura (`gc.disable()`) y aleatorización del orden de evaluación sobre 1,350 muestras temporales; la corrección matemática se valida exhaustivamente contra NetworkX y mediante un certificado formal de optimalidad de Bellman en tiempo $O(|V| + |E|)$.

Los resultados empíricos demuestran que no existe una estructura óptima universal. El 4-ary Heap obtiene la menor mediana en 7 de las 10 escalas sintéticas, logrando una razón mediana de 0.96 frente al heap binario en grafos dispersos y de 0.95 en grafos densos debido a la reducción del 50 % en la altura del árbol y la alineación con las líneas de caché L1/L2 de 64 bytes; sin embargo, los intervalos intercuartílicos (IQR) se solapan en todas las escalas. El heap binario resulta superior en 3 escalas sintéticas y en la consulta sobre la red real de Cusco (15.65 ms frente a 19.87 ms del 4-ary y 26.09 ms del Fibonacci), favorecido por su nula sobrecarga de indirección en grafos moderados. El Fibonacci Heap es superado sistemáticamente (entre 1.05 y 1.59 veces más lento que el binario), evidenciando que su cota asintótica $O(1)$ en `Decrease-Key` es eclipsada por la constante oculta de alocación dinámica de punteros. La extensión con búsqueda heurística A^* reduce las expansiones de vértices en un 7.2 % preservando la optimalidad. Se concluye que la selección de estructuras de datos viales debe fundamentarse en la densidad, la escala y la jerarquía de memoria física antes que en cotas asintóticas puras.

Palabras clave: Algoritmo de Dijkstra, colas de prioridad, heap binario, heap 4-ario, Fibonacci heap, rutas mínimas, OpenStreetMap, Cusco, A^*, análisis de algoritmos.

Abstract

This project addresses the efficient computation of shortest paths in urban road networks through Dijkstra's Algorithm instrumented with three priority queue data structures: an Indexed Binary Min-Heap, an indexed 4-ary Heap ($d = 4$) optimized for cache locality, and an amortized Fibonacci Heap. The research integrates an applied real-world case study on the Cusco road network retrieved from OpenStreetMap (891 vertices and 2,139 directed edges), evaluated under a generalized cost function incorporating Andean topography and longitudinal road gradients, along with controlled experimentation on reproducible synthetic instances with sparse ($|E| = 4|V|$) and dense ($|E| = 0.08\,|V|(|V|-1)$) profiles across ten scale tiers. The measurement protocol enforces a nanosecond timer (`perf_counter_ns`), explicit garbage collection isolation (`gc.disable()`), and randomized execution order across 1,350 wall-clock samples; mathematical correctness is verified against NetworkX and through a formal Bellman optimality certificate in $O(|V|+|E|)$ time.

Empirical results demonstrate that no universally optimal priority queue exists. The 4-ary Heap achieves the lowest median execution time in 7 out of 10 synthetic scales, with a median ratio of 0.96 relative to the binary heap in sparse graphs and 0.95 in dense graphs due to a 50% reduction in tree depth and alignment with 64-byte L1/L2 cache lines; however, interquartile ranges (IQR) overlap across all scales. The binary heap proves superior in 3 synthetic scales and in the real Cusco network query (15.65 ms versus 19.87 ms for the 4-ary heap and 26.09 ms for the Fibonacci heap), driven by zero pointer indirection overhead in moderate-scale topologies. The Fibonacci Heap is consistently outperformed (1.05 to 1.59 times slower than the binary heap), confirming that its theoretical $O(1)$ amortized bound for `Decrease-Key` is eclipsed by dynamic memory allocation constants. The A^* heuristic extension achieves a 7.2% reduction in vertex expansions while preserving exact optimality. We conclude that priority queue selection for road networks must prioritize graph density, scale, and physical memory hierarchy over pure asymptotic bounds.

Keywords: Dijkstra's algorithm, priority queues, binary heap, 4-ary heap, Fibonacci heap, shortest paths, OpenStreetMap, Cusco, A^*, experimental analysis.

Índice

1. Resumen I
Abstract . . . II

2. Introducción 1
2.1. Contexto y Motivación . . . 1
2.2. Planteamiento del Problema . . . 1
2.3. Objetivos del Proyecto . . . 2
2.4. Alcance y Delimitación . . . 2
2.5. Control de Versiones y Repositorio Oficial en GitHub . . . 3
2.6. Organización del Documento . . . 3

3. Fundamentos y Aprendizaje Autónomo 4
3.1. Conceptos y Algoritmos Requeridos . . . 4
3.2. Fuentes Estudiadas . . . 6
3.3. Conocimientos Nuevos y Brechas Técnicas . . . 6
3.4. Estrategia y Evidencias de Aprendizaje Autónomo . . . 7

4. Evaluación y Selección del Stack Tecnológico 8
4.1. Alternativas Consideradas . . . 8
4.2. Criterios de Comparación . . . 8
4.3. Tecnología Seleccionada y Justificación . . . 8
4.4. Procedimiento de Aprendizaje e Integración . . . 9
4.5. Dificultades de Adopción y Soluciones Aplicadas . . . 9

5. Formulación del Problema 11
5.1. Definición Formal del Grafo Vial Urbano . . . 11
5.2. Parámetros de Entrada, Variables de Decisión y Salidas del Sistema . . . 11
5.3. Restricciones de Conservación de Flujo y No Negatividad . . . 12
5.4. Función Objetivo y Modelo de Costo con Impedancia Andina . . . 12
5.5. Ejemplo Ilustrativo Resuelto Analíticamente (Red Nuclear de 6 Nodos) . . . 14
5.6. Traza Manual Paso a Paso del Algoritmo de Dijkstra ($k = 0$ a $k = 6$) . . . 14
5.7. Instancia de Verificación Canónica: CLRS Figura 24.6 . . . 16
5.8. Red Vial Urbana Real de Cusco (OpenStreetMap) . . . 16

6. Diseño Algorítmico 17
6.1. Algoritmo de Dijkstra con Cola de Prioridad Abstracta . . . 17
6.2. Invariantes de Bucle y Demostración Formal de Corrección . . . 18
6.3. Estructuras de Datos de Cola de Prioridad Evaluadas . . . 19
6.4. Análisis Comparativo de Complejidades Teóricas . . . 20
6.5. Guía Práctica de Selección según el Escenario . . . 20

7. Diseño e Implementación del Sistema 22
7.1. Arquitectura Modular del Sistema (4 Capas) . . . 22
7.2. Descripción Detallada de Módulos y Componentes . . . 22

7.3. Formatos y Estructura de Datos (Serialización JSON) . . . 23
7.4. Contratos de Interfaz e Inyección de Colas de Prioridad . . . 24
7.5. Manejo Defensivo de Entradas y Validación . . . 24
7.6. Servicios Complementarios: Meteorología y Escenarios de Tráfico . . . 24
7.7. Decisiones de Implementación Relevantes y Justificación Técnica . . . 25

8. Estrategia de Pruebas **26**
8.1. Framework de Pruebas y Patrón Arrange-Act-Assert (AAA) . . . 26
8.2. Tipología y Cobertura de la Suite Automatizada (278 Pruebas) . . . 26
8.3. Metodología de Verificación y Oráculos Independientes . . . 27
8.4. Certificado Formal de Optimalidad de Bellman $O(|V|+|E|)$ . . . 28
8.5. Casos Límite, Condiciones Adversas y Pruebas de Regresión . . . 28
8.6. Pruebas de Integración y Funcionamiento Desconectado . . . 28
8.7. Criterios de Aceptación y Calidad del Software . . . 29

9. Metodología Experimental **30**
9.1. Preguntas de Investigación e Hipótesis de Evaluación . . . 30
9.2. Diseño Factorial e Instancias Experimentales . . . 30
9.3. Variables Experimentales y Métricas de Desempeño . . . 31
9.4. Entorno Experimental Controlado e Instrumentación de Precisión . . . 31
9.5. Caso Aplicado sobre la Red Vial Urbana Real de Cusco . . . 32
9.6. Procedimiento Experimental Reproducible . . . 33
9.7. Amenazas a la Validez . . . 33

10. Resultados **34**
10.1. Verificación de Corrección y Equivalencia de Distancias . . . 34
10.2. Benchmark Canónico en Instancias Sintéticas (Perfiles Disperso y Denso) . . . 34
10.3. Análisis Cuantitativo de Operaciones Elementales y Comparaciones . . . 36
10.4. Caso de Estudio Aplicado: Red Vial Real de Cusco (OpenStreetMap) . . . 37
10.5. Extensión Algorítmica con Búsqueda Heurística A^* . . . 39
10.6. Análisis de Sensibilidad Meteorológica y Tráfico . . . 39

11. Discusión Crítica **40**
11.1. Contraste entre Teoría Asintótica y Desempeño Empírico . . . 40
11.2. Análisis del Comportamiento del Montículo 4-ario y Localidad de Caché . . . 40
11.3. Desempeño del Montículo Binario en la Red Real de Cusco . . . 41
11.4. Influencia de la Densidad del Grafo y Operaciones Decrease-Key . . . 41
11.5. Evaluación de la Poda Espacial con Búsqueda Heurística A^* . . . 41
11.6. Análisis de Sensibilidad de Factores Ambientales e Invariancia de Ruta . . . 41
11.7. Guía Práctica y Marco de Decisión para Selección de Colas . . . 42
11.8. Limitaciones del Estudio y Amenazas a la Validez . . . 42

12. Conclusiones y Recomendaciones **43**
12.1. Balance de Objetivos Específicos (OE1–OE4) . . . 43
12.2. Respuesta a las Preguntas de Investigación . . . 43

12.3. Conclusión Principal y Síntesis de Contribuciones 44
12.4. Recomendaciones para la Práctica de Ingeniería 44
12.5. Líneas de Trabajo Futuro y Posibles Extensiones 44
12.6. Lecciones Aprendidas 45

13. Referencias **46**

14. Anexos **47**
Anexo A: Instalación y configuración 47
Anexo B: Uso del prototipo 47
Anexo C: Cronología verificable 48
Anexo D: Matriz de trazabilidad 49
Anexo E: Matriz de contribuciones 50
Anexo F: Estado del seguimiento 50

Índice de figuras

1. Instancia didáctica del Centro Histórico de Cusco ($|V| = 6$, $|E| = 9$). Longitudes esquemáticas en metros. . . . 14
2. Arquitectura modular desacoplada del sistema en cuatro capas operativas . . . 22
3. Tiempo mediano de ejecución de Dijkstra con las tres colas de prioridad evaluadas. Izquierda: perfil disperso ($|E| = 4|V|$). Derecha: perfil denso ($|E| = 0.08\,|V|(|V| - 1)$). Las barras de error representan el Intervalo Intercuartílico (IQR). . . . 35
4. Distribución de operaciones elementales de Dijkstra en el perfil disperso sintético. Se discriminan inserciones, extracciones, decrementos de clave efectivos e intentos totales de relajación. . . . 37
5. Ruta mínima óptima calculada sobre la red vial OpenStreetMap del Cusco. La línea resalta el itinerario de emergencia y los marcadores señalan el origen y destino evaluados. 38

Índice de tablas

1. Selección razonada de fuentes bibliográficas clave y criterios de pertinencia . . . 6
2. Alternativas por capa tecnológica . . . 8
3. Matriz multicriterio del lenguaje principal (escala 1–5) . . . 8
4. Velocidades nominales por tipología de vía en Cusco . . . 13
5. Tramos viales dirigidos, distancias métricas y costo de impedancia en el Cusco. . . . 14
6. Traza matricial de distancias mínimas y predecesores en cada iteración del algoritmo. . . . 15
7. Resultados computacionales de distancias y tiempos óptimos calculados desde Plaza Mayor 16
8. Correspondencia entre pseudocódigo y la implementación en Python . . . 18
9. Cotas temporales y espaciales de las colas de prioridad evaluadas . . . 20
10. Guía de selección práctica de algoritmos y estructuras en redes viales . . . 21
11. Archivos de datos principales y su rol funcional en el sistema . . . 23
12. Cobertura de las pruebas automatizadas por categoría funcional . . . 26
13. Distribución modular de la suite de 278 pruebas . . . 27
14. Especificaciones del entorno experimental de medición . . . 32
15. Dijkstra con tres colas, perfil disperso: mediana e IQR en μs. . . . 34
16. Dijkstra con tres colas, perfil denso: mediana e IQR en μs. . . . 35
17. Síntesis comparativa por perfil y consulta. Las razones usan la mediana; no se declara un ganador cuando los IQR se solapan. . . . 36
18. Caso OSM: tiempo de Dijkstra por cola, mediana e IQR. . . . 38
19. Sensibilidad meteorológica uniforme en la consulta OSM. . . . 39
20. Cronología de aportes individuales comprobables en Git . . . 49
21. Trazabilidad entre objetivos, componentes y evidencia . . . 49
22. Contribuciones por integrante y evidencia en el repositorio . . . 50

2. Introducción

2.1. Contexto y Motivación

La determinación eficiente de rutas mínimas en redes viales urbanas representa uno de los problemas fundamentales y de mayor impacto práctico en la ciencia de la computación, la ingeniería de transporte y la logística de atención a emergencias. En entornos metropolitanos contemporáneos, los sistemas de despacho para ambulancias, unidades de bomberos y patrullaje policial requieren calcular trayectorias óptimas en fracciones de segundo para minimizar los tiempos de respuesta, donde reducciones marginales de tiempo se traducen directamente en la preservación de vidas humanas.

La ciudad del Cusco, ubicada en la sierra sur del Perú a una altitud media de 3,399 m s. n. m., presenta una morfología urbana singular. Su Centro Histórico, declarado Patrimonio de la Humanidad por la UNESCO en 1983, superpone una traza urbana incaica y colonial caracterizada por calzadas sumamente angostas, pasajes adoquinados, alta densidad de giros obligatorios y severas pendientes topográficas que oscilan con frecuencia entre el 5 % y más del 15 %. En este contexto geográfico, la distancia euclidiana o la longitud puramente física de un tramo vial resultan insuficientes como métrica de optimización. Un trayecto físicamente más corto pero con un gradiente positivo pronunciado impone un esfuerzo motriz severo sobre un vehículo pesado de emergencia, reduciendo drásticamente su velocidad operativa y aumentando el riesgo de demoras críticas. Por ende, resulta imperativo formular modelos de costo generalizado que integren la topografía andina y evaluar algoritmos de caminos mínimos capaces de procesar mallas viales a gran escala con alta eficiencia temporal y espacial.

2.2. Planteamiento del Problema

El Algoritmo de Dijkstra (Dijkstra, 1959) es el estándar canónico para resolver el problema de caminos mínimos desde un origen único (*Single-Source Shortest Path*, SSSP) en grafos dirigidos con pesos no negativos ($w(e) \geq 0$). La complejidad asintótica de este algoritmo depende intrínsecamente de la estructura de datos utilizada para implementar la cola de prioridad que gestiona los vértices tentativos:

$$T(|V|,|E|) = O(|V| \cdot T_{\text{Extract-Min}} + |E| \cdot T_{\text{Decrease-Key}} + |V| \cdot T_{\text{Insert}}) \tag{1}$$

En la literatura teórica clásica, el Fibonacci Heap propuesto por Fredman y Tarjan (1987) ostenta la cota asintótica óptima conocida de $O(|E| + |V| \log |V|)$, al ejecutar la operación de disminución de clave (`Decrease-Key`) en tiempo amortizado $O(1)$ y la extracción del mínimo (`Extract-Min`) en $O(\log |V|)$. No obstante, estudios computacionales de referencia (Cherkassky et al., 1996; LaMarca & Ladner, 1996) han reportado que, en redes dispersas donde $|E| = O(|V|)$ (como las redes viales donde el grado promedio por nodo rara vez supera 4), la constante oculta en la notación asintótica del Fibonacci Heap, originada por la alocación dinámica en memoria, la manipulación de múltiples punteros y los fallos en las líneas de caché (*cache misses*), degrada su rendimiento en tiempo real frente a montículos contiguos como el Binary Heap o el d-ary Heap.

El problema central que aborda este proyecto radica en investigar, implementar y contrastar cuantitativamente el comportamiento empírico y teórico de distintas colas de prioridad acopladas al algoritmo de Dijkstra, evaluando su desempeño real sobre la red vial urbana del Cusco bajo una función

de costo generalizado con impedancia topográfica.

2.3. Objetivos del Proyecto

Objetivo General

Diseñar, implementar y evaluar experimentalmente una solución computacional modular para el cálculo de rutas mínimas en la red vial urbana de la ciudad del Cusco, contrastando el desempeño en tiempo de ejecución, operaciones elementales y consumo de memoria del Algoritmo de Dijkstra instrumentado con tres estructuras de colas de prioridad: Min-Heap Binario Indexado, d-ary Heap y Fibonacci Heap, verificando formalmente la optimalidad de las soluciones.

Objetivos Específicos

Siguiendo la metodología de ingeniería de software y análisis de algoritmos, los objetivos específicos se ordenan en cuatro fases secuenciales e integradas:

1. **OE1 (Modelar):** Formular el modelo matemático del grafo vial dirigido del Centro Histórico del Cusco y su extensión a la red metropolitana OpenStreetMap, definiendo una función de costo generalizado (impedancia andina) que cuantifique el impacto de calles de un solo sentido, calles angostas coloniales, pasajes peatonales y gradientes altitudinales positivos.

2. **OE2 (Diseñar e Implementar):** Diseñar e implementar bajo una arquitectura desacoplada en cuatro capas lógicas el algoritmo de Dijkstra parametrizado, integrando tres alternativas de colas de prioridad puras con soporte formal para `Decrease-Key`: Min-Heap Binario Indexado, 4-ary Heap ($d = 4$) optimizado para líneas de caché L1/L2, y Fibonacci Heap amortizado, extendiendo el sistema hacia búsqueda heurística A^*.

3. **OE3 (Verificar):** Validar analíticamente la correctitud del algoritmo mediante traza manual paso a paso, certificado formal de optimalidad independiente en $O(|V| + |E|)$ y una suite automatizada de pruebas unitarias bajo el patrón Arrange-Act-Assert (AAA), contrastando los resultados contra oráculos canónicos de referencia (NetworkX y Bellman-Ford).

4. **OE4 (Evaluar e Investigar):** Evaluar experimentalmente la escalabilidad asintótica, el consumo de operaciones elementales y los tiempos de CPU en redes viales sintéticas controladas y en la red vial real OSM de Cusco, contrastando la teoría asintótica clásica frente a la localidad espacial de memoria caché en hardware contemporáneo.

2.4. Alcance y Delimitación

El proyecto se delimita formalmente en las siguientes dimensiones:

- **Delimitación Espacial:** Se analizan dos escalas complementarias: (1) La malla vial nuclear del Centro Histórico del Cusco (6 intersecciones estratégicas: Plaza Mayor, Plaza Regocijo, Qorikancha, Plaza San Francisco, Mercado Central San Pedro y Plazoleta Limacpampa), empleada para traza

manual analítica y verificación paso a paso; y (2) La red vial urbana real de Cusco extraída de OpenStreetMap vía OSMnx (Boeing, 2017), con un radio de 1,500 m, 891 vértices y 2,139 aristas dirigidas.

- **Delimitación Algorítmica:** Se evalúa el algoritmo de Dijkstra en grafos dirigidos estáticos con costos deterministas no negativos ($w(e) \geq 0$). Como extensión de ruteo punto a punto se integra A^* con heurística consistente. Bellman-Ford y NetworkX operan exclusivamente como oráculos independientes de verificación cruzada.
- **Alcance del Sistema:** Comprende la formulación matemática integral, la solución analítica manual, el núcleo ejecutable en Python 3.12 con interfaz de línea de comandos (CLI), la aplicación interactiva web en Streamlit, el cuaderno didáctico en Google Colab y la suite automatizada de 278 pruebas con Pytest.

2.5. Control de Versiones y Repositorio Oficial en GitHub

Conforme a las directrices de reproducibilidad y trabajo colaborativo del curso, el desarrollo íntegro del proyecto se gestiona bajo el sistema de control de versiones Git y se aloja en el repositorio público oficial:

https://github.com/frankich99/PROYECTO_Algoritmos-Avanzados-UNSAAC

La organización de ramas adopta un esquema estructurado: la rama `main` almacena las versiones estables consolidadas, complementada por ramas de desarrollo temáticas por integrante (`ramaFRANKI`, `ramaAldo`, `ramaYeni`, `RamaRomario`). Los datos experimentales se versionan en `python/data/processed/` para garantizar reproducibilidad total sin requerir descargas en vivo de APIs externas.

2.6. Organización del Documento

El presente informe técnico se estructura estrictamente en catorce secciones canónicas conforme a los requerimientos del apartado 9 del sílabo: La Sección 2 presenta la introducción, contexto y objetivos. La Sección 3 detalla los fundamentos teóricos, colas de prioridad, búsqueda heurística A^* y el proceso de aprendizaje autónomo. La Sección 4 evalúa y justifica el stack tecnológico mediante una matriz multicriterio. La Sección 5 expone la formulación matemática del problema y el modelo de costo cusqueño. La Sección 6 formaliza el diseño algorítmico, pseudocódigos, invariantes y complejidades. La Sección 7 detalla la arquitectura modular del software y los componentes implementados. La Sección 8 especifica la estrategia de pruebas unitarias (AAA), oráculos y certificado de optimalidad. La Sección 9 formula las hipótesis, variables y el protocolo de experimentación reproducible. La Sección 10 documenta los resultados computacionales del benchmark canónico y la red real OSM. La Sección 11 analiza críticamente los hallazgos empíricos y el impacto de la jerarquía de memoria. La Sección 12 sintetiza las conclusiones en respuesta a los objetivos planteados y propone trabajo futuro. La Sección 13 lista las fuentes académicas bajo norma APA 7ma edición. Finalmente, la Sección 14 recopila manuales de instalación y uso, bitácoras y matrices de contribución.

3. Fundamentos y Aprendizaje Autónomo

3.1. Conceptos y Algoritmos Requeridos

El desarrollo del proyecto descansa sobre tres pilares conceptuales de las ciencias de la computación: la teoría de grafos viales dirigidos, el diseño de colas de prioridad con clave disminuible y la relajación voraz (*greedy*) de caminos mínimos.

Grafos Viales Urbanos y Propiedad de Red Plana

Una red vial urbana se modela formalmente como un grafo dirigido ponderado $G = (V, E, W)$, donde V representa el conjunto de intersecciones viales o discontinuidades geométricas, $E \subseteq V \times V$ representa los tramos viales orientados según el sentido de circulación reglamentario, y $W : E \to \mathbb{R}^+$ asigna un costo no negativo a cada tramo. Las redes viales urbanas presentan propiedades geométricas cuasi-planas con una densidad extremadamente baja:

$$|E| \approx k \cdot |V|, \quad \text{donde } 2.0 \leq k \leq 3.5 \tag{2}$$

En el recorte vial de OpenStreetMap de Cusco procesado en este proyecto ($|V| = 891, |E| = 2139$), la razón es $|E|/|V| = 2.40$. Esta dispersión estructural condiciona de forma determinante el comportamiento asintótico y práctico de los algoritmos de optimización de rutas.

Algoritmo de Dijkstra e Invariante de Subestructura Óptima

El algoritmo de Dijkstra (Dijkstra, 1959) explora el espacio de búsqueda manteniendo una partición de los vértices en dos conjuntos disjuntos: el conjunto S de vértices cuya distancia mínima desde el origen s ya ha sido permanentemente determinada, y el conjunto $Q = V \setminus S$ de vértices tentativos gestionados en una cola de prioridad. En cada paso iterativo, el algoritmo extrae el vértice $u \in Q$ con la menor distancia tentativa acumulada $d[u]$, lo incorpora a S, y relaja cada arista saliente $(u, v) \in E$:

$$\text{si } d[u] + w(u,v) < d[v] \implies d[v] \leftarrow d[u] + w(u,v), \quad \pi[v] \leftarrow u \tag{3}$$

La actualización del valor $d[v]$ en la cola de prioridad exige ejecutar la operación `Decrease-Key`$(Q, v, d[v])$.

Estructuras de Colas de Prioridad con Decrease-Key

La cota asintótica de Dijkstra está dominada por las tres primitivas de la cola:

- **Min-Heap Binario Indexado:** Árbol binario casi completo representado en un arreglo contiguo en memoria, complementado por una tabla hash inversa de posiciones $\text{pos} : V \to \mathbb{N}$ que ubica cualquier vértice en $O(1)$. `Insert` toma $O(\log |V|)$, `Extract-Min` toma $O(\log |V|)$ y `Decrease-Key` toma $O(\log |V|)$.
- **4-ary Heap Contiguo ($d = 4$):** Árbol d-ario con factor de ramificación 4. Cada nodo padre en el

índice i tiene sus 4 hijos en los índices $4i+1, 4i+2, 4i+3, 4i+4$. Esto reduce la profundidad del árbol a la mitad ($\log_4 |V| = 0.5 \log_2 |V|$), requiriendo menos niveles de flotado en `Decrease-Key` y aprovechando las líneas de caché L1/L2 de 64 bytes.

- **Fibonacci Heap:** Colección de árboles con orden de montículo estructurados mediante listas circulares doblemente enlazadas con punteros `parent`, `child`, `left`, `right`, `degree` y `marked` (Fredman & Tarjan, 1987). Utiliza inserción y corte perezoso (*lazy*), permitiendo que `Insert` y `Decrease-Key` operen en $O(1)$ amortizado, y `Extract-Min` en $O(\log |V|)$ amortizado mediante consolidación de árboles de igual grado.

Extensión Algorítmica: Búsqueda Heurística A^* y Poda Espacial

Una limitación intrínseca de Dijkstra en consultas punto a punto ($s \to t$) es que expande nodos en forma de onda radial concéntrica en todas direcciones. El Algoritmo A^* (Hart et al., 1968) focaliza la búsqueda mediante una función de evaluación compuesta:

$$f(n) = g(n) + h(n) \tag{4}$$

donde $g(n)$ es el costo acumulado desde el origen s, y $h(n)$ es una heurística admisible y consistente que estima el costo residual hacia el destino t. Para garantizar optimalidad, la heurística debe satisfacer:

1. **Admisibilidad:** $0 \leq h(n) \leq h^*(n)$ para todo $n \in V$, donde $h^*(n)$ es el costo óptimo real.
2. **Consistencia (Monotonicidad):** $h(u) \leq w(u,v) + h(v)$ para toda arista dirigida $(u,v) \in E$.

En este proyecto, para la red vial cusqueña con pesos de tiempo, la heurística euclidiana se escala mediante el factor admisible:

$$\lambda = \min_{(u,v)\in E} \frac{w(u,v)}{\|x_u - x_v\|}, \quad h(u) = \lambda \cdot \|x_u - x_t\| \tag{5}$$

asegurando consistencia matemática rigurosa y podando expansiones sin alterar el costo mínimo óptimo.

Metodología de Pruebas Unitarias: El Patrón AAA

La totalidad de la suite de pruebas del proyecto (278 pruebas con Pytest) se implementa bajo el patrón **Arrange - Act - Assert (AAA)**:

- **Arrange:** Inicialización de precondiciones, generación determinista de instancias con semilla fija y selección de la cola a evaluar.
- **Act:** Invocación del solver (`dijkstra_solver` o `a_star_solver`) y captura de vectores de distancias y contadores de operaciones.
- **Assert:** Verificación estricta contra oráculos independientes (Bellman-Ford, NetworkX) y evaluación del certificado de optimalidad.

3.2. Fuentes Estudiadas

La selección de literatura se condujo con criterios de rigor académico, priorizando artículos seminales arbitrados por pares, manuales algorítmicos estándar de la Association for Computing Machinery (ACM) e IEEE, e investigaciones sobre análisis de rendimiento bajo jerarquía de memoria (Tabla 1).

Tabla 1
Selección razonada de fuentes bibliográficas clave y criterios de pertinencia

Fuente Académica	**Tipología y Dominio**	**Criterio de Pertinencia e Impacto en el Proyecto**
Dijkstra (1959)	Artículo Seminal (Numerische Mathematik)	Fundamento teórico del algoritmo SSSP con pesos no negativos y principio de relajación voraz.
Fredman & Tarjan (1987)	Artículo Teórico (Journal of the ACM)	Especificación formal del Fibonacci Heap con tiempos amortizados $O(1)$ para `Decrease-Key`.
Tarjan (1983)	Monografía Clásica (SIAM)	Análisis riguroso del método del potencial para el cálculo de costos amortizados en estructuras de datos.
Cherkassky et al. (1996)	Estudio Empírico (Journal of Algorithms)	Evaluación experimental exhaustiva de algoritmos de caminos mínimos; evidencia de ineficiencias del Fibonacci Heap en grafos dispersos.
LaMarca & Ladner (1996)	Arquitectura y Algoritmos (ACM SODA)	Análisis del impacto de las líneas de caché L1/L2 y fallos de traducción TLB en montículos binarios y d-arios.
Hart et al. (1968)	Artículo Seminal (IEEE SSC)	Formulación de A^* y condiciones de admisibilidad y consistencia para poda de grafos.
Boeing (2017)	Sistemas Geoespaciales (Computers, Env. and Urban Systems)	Metodología de ingestión, validación y corrección topológica de grafos urbanos a partir de OpenStreetMap.
Cormen et al. (2022)	Libro de Texto Avanzado (MIT Press)	Demostración formal de invariantes de bucle, cotas de recurrencia y pseudocódigo estructurado.

3.3. Conocimientos Nuevos y Brechas Técnicas

Al inicio del proyecto, el equipo diagnosticó cuatro brechas conceptuales y técnicas que trascendían los contenidos estándar de cursos formativos previos:

1. **Brecha 1: Manipulación de Estructuras Arbóreas con Punteros Múltiples.** Dominio de la representación en memoria de listas circulares doblemente enlazadas con punteros padre, hijo, izquierdo y derecho para el Fibonacci Heap, garantizando concatenaciones en $O(1)$ sin pérdida de referencias.

2. **Brecha 2: Operación Decrease-Key en Tiempo Constante Amortizado $O(1)$.** Comprensión analítica del mecanismo de marcado de nodos (*node marking*) y cortes en cascada (*cascading cuts*) propuesto por Tarjan, asegurando que el grado de cualquier nodo en un heap de n elementos no exceda $\lfloor \log_\phi n \rfloor$.

3. **Brecha 3: Arquitectura de Computadores, Jerarquía de Memoria y Líneas de Caché L1/L2.** Comprensión del fenómeno por el cual una estructura asintóticamente superior como el Fibonacci

Heap es superada en hardware contemporáneo por estructuras con peor cota teórica (d-ary heap) debido a la contigüidad espacial en los bloques de caché de 64 bytes.

4. **Brecha 4: Modelado de Redes Viales Reales con OpenStreetMap y Grafos Geoespaciales.** Extracción de mallas viales urbanas reales mediante OSMnx, conversión de primitivas geoespaciales WGS84 a grafos dirigidos ponderados y manejo de tramos viales paralelos y sentidos únicos.

3.4. Estrategia y Evidencias de Aprendizaje Autónomo

Para cerrar las brechas identificadas sin supervisión externa continua, el equipo adoptó una estrategia estructurada en cuatro hitos:

1. **Hito 1 (Investigación y Prototipo Temprano):** Lectura directa de Cormen et al. (2022) y Fredman y Tarjan (1987). Implementación de un prototipo temprano del Min-Heap binario indexado con tabla de posiciones $O(1)$ para validar la mecánica de `decrease_key`.

2. **Hito 2 (Extensión d-aria y Análisis de Caché):** Estudio de la investigación de LaMarca y Ladner (1996) e implementación del montículo 4-ario contiguo, verificando que la reducción de niveles en el árbol compensa el aumento de comparaciones por nivel.

3. **Hito 3 (Fibonacci Heap y Detección de Defectos):** Construcción manual del montículo de Fibonacci con punteros dobles circulares. Durante las pruebas de estrés, el equipo detectó de forma autónoma que un arreglo de consolidación con tamaño estático $\log_2(n)+7$ causaba pérdida de árboles; se corrigió analíticamente redimensionándolo dinámicamente con la cota de la razón áurea $\lceil \log_\phi(n) \rceil + 2$.

4. **Hito 4 (Validación Rigurosa):** Implementación del certificador de optimalidad de Bellman $O(|V|+|E|)$ y de la suite automatizada de 278 pruebas con Pytest, asegurando que las tres estructuras produzcan distancias matemáticamente idénticas en todas las instancias.

Las evidencias de este aprendizaje autónomo se encuentran registradas cronológicamente en los commits del repositorio de GitHub y en la suite de pruebas unitarias.

4. Evaluación y Selección del Stack Tecnológico

4.1. Alternativas Consideradas

La Tabla 2 resume las alternativas evaluadas en seis capas. Se distingue entre la tecnología adoptada para el prototipo y las opciones consideradas para una eventual ampliación. No se valoró la novedad por sí misma, sino la capacidad de aprender, integrar, probar y justificar cada herramienta.

Tabla 2
Alternativas por capa tecnológica

Capa	Alternativas	Decisión	Criterio principal
Lenguaje principal	C++20, Rust 1.75, Python 3.12	**Python 3.12**	Expresividad algorítmica, tipado, instrumentación y rapidez de experimentación.
Red vial y grafos	OSMnx/NetworkX, Overpass API, PostGIS	**OSMnx + NetworkX**	Descarga y conversión reproducible a grafo dirigido; NetworkX se usa como oráculo.
Pruebas	pytest, unittest, GoogleTest	**pytest**	Parametrización, fixtures, aserciones y compatibilidad directa con el código Python.
Interfaz web	Streamlit + Plotly, Dash, Flask + HTML	**Streamlit + Plotly**	Permite construir la aplicación web en Python con vistas interactivas y despliegue local sencillo. Dash y Flask no forman parte del prototipo.
Aplicación móvil	Flutter/Dart, React Native, Kivy	*No implementada*	Flutter/Dart queda como alternativa para una futura aplicación móvil; no se presenta como parte del sistema actual.
Documentación	LaTeX, Typst, Word	**LaTeX + BibLaTeX/Biber**	Ecuaciones, tablas, referencias APA y control de versiones.

4.2. Criterios de Comparación

Se usaron cuatro criterios ponderados: C1 expresividad algorítmica (30 %), C2 instrumentación y aislamiento del ruido (25 %), C3 reproducibilidad y portabilidad (25 %) y C4 curva de aprendizaje (20 %). La Tabla 3 presenta una escala ordinal de 1 a 5. Los puntajes son un juicio técnico del equipo, no una medición física; se incluyen para hacer explícita la decisión, no para simular precisión inexistente.

Tabla 3
Matriz multicriterio del lenguaje principal (escala 1–5)

Alternativa	C1 (30 %)	C2 (25 %)	C3 (25 %)	C4 (20 %)	Ponderado
Python 3.12	5.0	4.2	4.8	4.8	**4.71**
C++20	3.8	4.8	3.5	2.8	3.78
Rust 1.75	4.0	4.6	3.6	2.2	3.69

4.3. Tecnología Seleccionada y Justificación

1. **Python 3.12.8.** Permite expresar los algoritmos con tipado y pruebas sin una cadena de compilación. Las tres colas se implementaron desde cero; no se usó `heapq` para las estructuras evaluadas.

La contrapartida es que los tiempos absolutos incluyen la sobrecarga del intérprete y no son extrapolables a C++ o Rust.

2. **OSMnx 2.1.1 y NetworkX 3.7.** OSMnx descarga y convierte la red vial; NetworkX se usa como oráculo de distancias. La red se congela en JSON para que el análisis no dependa de una descarga en vivo.

3. **pytest 9.1.1.** Pruebas parametrizadas, fixtures y ejecución selectiva. La suite completa se ejecuta con `python -m pytest python/tests -q`.

4. **Streamlit 1.64.0 y Plotly 7.1.0.** Constituyen la interfaz web actual del proyecto: una sola aplicación para comparar resultados, explorar la ruta OSM, revisar la traza manual y consultar la procedencia de los datos. Dash y Flask + HTML se consideraron alternativas, pero no se implementaron. La interfaz es un medio de interacción; el núcleo evaluado sigue siendo el algoritmo.

5. **Flutter/Dart como alternativa móvil futura.** No se desarrolló una aplicación móvil en este trabajo. Si el sistema se amplía a Android y iOS, Flutter permitiría reutilizar el servicio y construir una interfaz multiplataforma; por tanto, se registra como trabajo futuro y no como tecnología usada en los resultados.

6. **LaTeX con BibLaTeX y Biber.** Informe reproducible, ecuaciones y referencias APA. La compilación se automatiza con `compilar.ps1`.

7. **Servicios meteorológicos (Open-Meteo y OpenWeatherMap API).** Se implementó un diseño de doble proveedor en `src/services/weather.py`. Como opción sin credenciales se emplea Open-Meteo (datos de modelo). Como proveedor alternativo autenticado se integró `OpenWeatherMapClient` (https://home.openweathermap.org/users/sign_up). Ambas fuentes alimentan `WeatherObservation`; las consultas opcionales de la web son temporales y no reescriben el caso canónico. Una clave debe suministrarse localmente, nunca en archivos versionados.

8. **Modelo horario de tráfico.** Reglas deterministas y coeficientes hipotéticos por franja y tipo de vía; la interfaz muestra los factores, pero aplica solo el de avenida de forma uniforme a la ruta OSM. El cliente opcional de TomTom no dispone de un experimento canónico ni de pruebas de red en este repositorio; no se declara tráfico calibrado.

4.4. Procedimiento de Aprendizaje e Integración

La adopción se organizó en tres fases. Primero se construyó un prototipo mínimo con el heap binario y la instancia de seis nodos. Después se añadieron el heap 4-ario, el Fibonacci heap y el certificado de optimalidad. Finalmente se integraron OSMnx, el benchmark canónico, la aplicación Streamlit y el servicio meteorológico con caché. Cada fase dejó pruebas y datos versionados; las decisiones que cambiaron el diseño se registraron en el repositorio.

4.5. Dificultades de Adopción y Soluciones Aplicadas

- **Dependencias geoespaciales en Windows.** La instalación de GDAL y OSMnx puede fallar por binarios incompatibles. Se exportó la red a JSON; las pruebas usan el *snapshot*, no la descarga.

`requirements.txt` declara versiones mínimas, no versiones exactas; el entorno de la medición está anotado en la Sección 9.

- **Codificación de consola.** La consola CP1252 producía errores con caracteres Unicode. Se forzó UTF-8 en archivos y se usaron mensajes ASCII en los registros de la CLI.
- **Reproducibilidad temporal.** Las primeras mediciones usaban promedios y no aislaban el recolector de basura. Se adoptó `perf_counter_ns`, *warm-up*, `gc.disable()` durante la muestra y mediana/IQR.
- **Interfaz y estado.** Streamlit reejecuta el script en cada interacción. Se separaron cálculo y visualización; se cachea la carga del grafo y del benchmark por fecha de modificación y se mantiene el resultado de la consulta en sesión. Las consultas meteorológicas requieren una acción explícita y no se cachean como si fueran actuales.
- **Clima sin API oficial de SENAMHI y latencia de claves.** Al no existir una API pública libre de SENAMHI, se adoptó Open-Meteo y la integración con OpenWeatherMap. Para mitigar la latencia de 10 a 30 minutos en la activación de claves de OpenWeatherMap (error `HTTP 401`), el cliente notifica al usuario e implementa respaldo automático en *snapshots* locales.

5. Formulación del Problema

5.1. Definición Formal del Grafo Vial Urbano

El problema de cálculo de rutas mínimas en la red vial urbana del Cusco se modela formalmente como la determinación de caminos de costo acumulado mínimo sobre un grafo dirigido y ponderado:

$$G = (V, E, w) \tag{6}$$

donde:

- V es el conjunto finito de intersecciones viales, nodos semaforizados o cambios de geometría del trazado urbano.
- $E \subseteq V \times V$ es el conjunto de tramos viales dirigidos orientados según el sentido reglamentario de circulación vehicular. Cada par ordenado $(u, v) \in E$ indica que está permitido transitar desde la intersección u hacia la intersección v.
- $w : E \to \mathbb{R}_{\geq 0}$ es la función de costo aplicado o impedancia generalizada de viaje asignada a cada arista, cumpliendo estrictamente con la restricción de no negatividad:

$$w(u, v) \geq 0, \quad \forall (u, v) \in E \tag{7}$$

5.2. Parámetros de Entrada, Variables de Decisión y Salidas del Sistema

Para un origen designado $s \in V$ y un destino $t \in V$, el problema de camino mínimo consiste en hallar

$$\delta(s,t) = \min_{P \in \mathscr{P}(s,t)} \sum_{e \in P} w(e), \tag{8}$$

donde $\mathscr{P}(s,t)$ es el conjunto de todos los caminos dirigidos simples que conectan s con t.

Variables de decisión binarias. Bajo el enfoque de optimización en redes, se define para cada arista dirigida $(u, v) \in E$ una variable de flujo binaria:

$$x_{uv} = \begin{cases} 1, & \text{si el tramo vial } (u,v) \text{ forma parte del camino mínimo óptimo;} \\ 0, & \text{en caso contrario.} \end{cases} \tag{9}$$

Salidas del sistema. El sistema calcula dos estructuras fundamentales:

1. El vector de distancias mínimas definitivas $d : V \to \mathbb{R}_{\geq 0} \cup \{\infty\}$, donde $d[v] = \delta(s, v)$ representa el costo óptimo desde s hasta v.
2. El vector de predecesores $\pi : V \to V \cup \{\text{NIL}\}$, que permite reconstruir de forma invertida la secuencia única de intersecciones del camino óptimo:

$$P(s,t) = \langle s = v_0, v_1, v_2, \ldots, v_k = t \rangle \tag{10}$$

5.3. Restricciones de Conservación de Flujo y No Negatividad

El flujo unitario inyectado en el origen s y absorbido en el destino t debe satisfacer el principio de balance de conservación de flujo en cada vértice $u \in V$:

$$\sum_{v:(u,v)\in E} x_{uv} - \sum_{v:(v,u)\in E} x_{vu} = \begin{cases} 1, & u = s \text{ (fuente)}, \\ -1, & u = t \text{ (sumidero)}, \\ 0, & u \in V \setminus \{s,t\} \text{ (nodos de tránsito intermedio)}, \end{cases} \tag{11}$$

sujeto a las restricciones de integridad y no negatividad de costos:

$$x_{uv} \in \{0,1\}, \quad w(u,v) \geq 0, \quad \forall (u,v) \in E \tag{12}$$

La condición $w(u,v) \geq 0$ garantiza que el algoritmo de Dijkstra sea rigurosamente aplicable, puesto que la subestructura óptima se preserva y los ciclos dirigidos tienen costo no negativo, garantizando que ningún camino elemental contenga bucles.

5.4. Función Objetivo y Modelo de Costo con Impedancia Andina

La función objetivo minimiza el costo total acumulado del recorrido:

$$\min Z = \sum_{(u,v)\in E} w(u,v)\, x_{uv} \tag{13}$$

Formulación de la impedancia vial andina. Es metodológicamente imprescindible distinguir tres escalas de instancias: la red manual didáctica de seis nodos (cuyas pendientes y longitudes son supuestos didácticos del equipo), el benchmark sintético (con pesos generados), y la consulta sobre la red real de OpenStreetMap (cuyos pesos proceden del *snapshot* congelado). Para modelar la circulación vehicular en el contexto topográfico cusqueño, se formula el siguiente modelo de impedancia generalizada:

$$w(u,v) = \frac{L(u,v)}{v_{\text{nom}}(\tau(u,v))} \cdot \Big(1 + \alpha \max\big(0, \theta(u,v)\big)\Big) \cdot m_{\text{clima}} \cdot c_{\text{tráfico}}(\tau(u,v)), \tag{14}$$

donde:

- $L(u,v)$ es la longitud física del segmento vial en metros.
- $\tau(u,v)$ representa la tipología de la vía (avenida, calle centro, calle residencial, peatonal restringida).
- $v_{\text{nom}}(\tau)$ es la velocidad nominal de diseño reglamentaria (Tabla 4).
- $\theta(u,v)$ es la pendiente longitudinal expresada como fracción decimal (por ejemplo, 0.03 para una gradiente del 3 %).
- $\alpha = 2.5$ es el coeficiente de penalización por esfuerzo motriz en pendiente ascendente. El operador $\max(0,\theta)$ garantiza que los descensos no descuenten tiempo de forma irreal.
- $m_{\text{clima}} \geq 1$ es el factor de sensibilidad meteorológica.

- $c_{\text{tráfico}}(\tau) \geq 1$ es el coeficiente de congestión vehicular.

Tabla 4
Velocidades nominales por tipología de vía en Cusco

Tipo de vía	Velocidad (km/h)	Velocidad (m/s)
Avenida	35	9.72
Calle residencial	25	6.94
Calle de centro histórico	15	4.17
Peatonal restringida	8	2.22

Variables meteorológicas. El multiplicador m_{clima} procesa variables atmosféricas mediante el modelo de Open-Meteo consultado para las coordenadas del Cusco $(-13.5165, -71.9788)$:

$$m_{\text{clima}} = 1 + c_p \,\text{mín}\left(\frac{P}{P_{\text{ref}}}, 1\right) + c_h \,\text{máx}\left(0, \frac{H - H_0}{100 - H_0}\right) + c_v \left(1 - \text{mín}\left(\frac{V}{V_{\text{ref}}}, 1\right)\right) + c_w \,\text{mín}\left(\frac{W}{W_{\text{ref}}}, 1\right), \tag{15}$$

donde P es la precipitación acumulada en mm ($P_{\text{ref}} = 10$ mm, $c_p = 0.20$), H es la humedad relativa en % ($H_0 = 65\,\%$, $c_h = 0.08$), V es la visibilidad horizontal en metros ($V_{\text{ref}} = 10\,000$ m, $c_v = 0.15$) y W es la ráfaga de viento en km/h ($W_{\text{ref}} = 80$ km/h, $c_w = 0.10$). La observación histórica canónica guardada en `applied_results.json` (28/10/2026 a las 12:30 hora de Lima) registra $P = 0.1$ mm, $V = 8920$ m y $W = 40.7$ km/h, produciendo un factor de escenario de $m_{\text{clima}} = 1.0691$ (+6.91 %).

Modelo de congestión horaria hipotético. El servicio `traffic.py` formula multiplicadores por franja y tipo de vía según supuestos analíticos:

$$c_{\text{tráfico}}(\tau) = 1 + \big(c_{\text{base}}(t) - 1\big) \cdot s(\tau) \cdot \gamma_{\text{sinergia}}, \tag{16}$$

donde $c_{\text{base}}(t) \in \{1.00, 1.25, 1.65, 2.20\}$ modela la hora pico; $s(\tau)$ pondera la capacidad vial (1.35 para calle céntrica, 1.15 para avenida, 0.95 para residencial); y $\gamma_{\text{sinergia}} = 1 + 0.20(m_{\text{clima}} - 1)$. En la consulta sobre OpenStreetMap, para evitar sesgos no calibrados en campo, la red opera con los tiempos de viaje base del *snapshot* (`travel_time_sec`) y el factor de avenida se evalúa como multiplicador global de sensibilidad.

Supuestos del modelo. (i) Los costos del benchmark y de la red OSM son estrictamente deterministas y no negativos. (ii) Las velocidades nominales representan límites operacionales seguros de diseño. (iii) La penalización por gradiente se aplica exclusivamente a pendientes positivas ($\theta > 0$). (iv) Los factores de clima y tráfico constituyen escenarios de sensibilidad uniforme que no alteran el orden topológico de las rutas en la red OSM. (v) La instancia de seis nodos es una abstracción docente para verificación analítica manual.

5.5. Ejemplo Ilustrativo Resuelto Analíticamente (Red Nuclear de 6 Nodos)

Para validar analíticamente el comportamiento del algoritmo, se diseñó la instancia representativa del Centro Histórico del Cusco mostrada en la Figura 1 y formalizada en la Tabla 5, con $|V| = 6$ intersecciones y $|E| = 9$ tramos viales dirigidos.

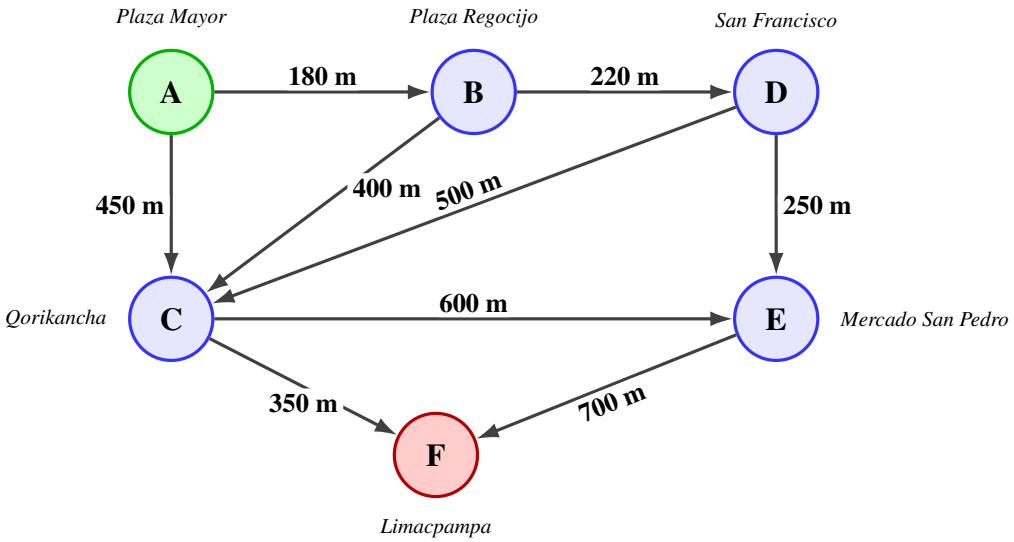


Figura 1
Instancia didáctica del Centro Histórico de Cusco ($|V| = 6$, $|E| = 9$). Longitudes esquemáticas en metros.

Tabla 5
Tramos viales dirigidos, distancias métricas y costo de impedancia en el Cusco.

Origen	Destino	Vía Urbana Asociada	Distancia (m)	Pendiente (%)	Tiempo Est. (s)
A	B	Calle Espaderos / Portal Carnicerías	180	+1.5	44.8
A	C	Calle Loreto / Pampa del Castillo	450	-2.0	108.0
B	D	Calle Garcilaso / Calle Marqués	220	+3.0	56.8
B	C	Calle San Bernardo / Almagro	400	-1.0	96.0
C	F	Calle Ahuacpinta / Limacpampa Grande	350	+0.5	36.5
C	E	Calle Matará / Tres Cruces de Oro	600	+2.0	151.2
D	E	Calle Santa Clara / Arco Santa Clara	250	+2.5	63.8
D	C	Calle Marqués / Mantas	500	-2.5	120.0
E	F	Calle Nueva Baja / Av. Tullumayo	700	-1.0	72.0

Nota. Tiempo de viaje estimado según la función de costo e impedancia por pendiente andina ($\alpha = 2.5$).

5.6. Traza Manual Paso a Paso del Algoritmo de Dijkstra ($k = 0$ a $k = 6$)

A continuación, se desarrolla analíticamente la traza paso a paso desde el origen $s = \text{A}$ (Plaza Mayor) bajo la métrica métrica de distancia física para verificación determinista:

- **Inicialización ($k = 0$):** $d[\text{A}] = 0$, $d[v] = \infty$ para todo $v \neq \text{A}$, $\pi[v] = \text{NIL}$, $S = \emptyset$. La cola contiene todos los vértices: $Q = \{(\text{A},0),(\text{B},\infty),(\text{C},\infty),(\text{D},\infty),(\text{E},\infty),(\text{F},\infty)\}$.

- **Paso 1 ($k=1$):** Se extrae $u=\mathrm{A}$ ($d[\mathrm{A}]=0$). Vértice permanente: $S=\{\mathrm{A}\}$. Relajación de aristas salientes:
  - (A,B): $d[\mathrm{A}]+180=180<d[\mathrm{B}](\infty) \implies d[\mathrm{B}]\leftarrow 180$, $\pi[\mathrm{B}]\leftarrow \mathrm{A}$, `Decrease-Key`$(Q,\mathrm{B},180)$.
  - (A,C): $d[\mathrm{A}]+450=450<d[\mathrm{C}](\infty) \implies d[\mathrm{C}]\leftarrow 450$, $\pi[\mathrm{C}]\leftarrow \mathrm{A}$, `Decrease-Key`$(Q,\mathrm{C},450)$.
- **Paso 2 ($k=2$):** Se extrae $u=\mathrm{B}$ ($d[\mathrm{B}]=180$). $S=\{\mathrm{A},\mathrm{B}\}$. Relajaciones:
  - (B,D): $180+220=400<d[\mathrm{D}](\infty) \implies d[\mathrm{D}]\leftarrow 400$, $\pi[\mathrm{D}]\leftarrow \mathrm{B}$, `Decrease-Key`$(Q,\mathrm{D},400)$.
  - (B,C): $180+400=580\not< d[\mathrm{C}](450)$. Se descarta sin cambio.
- **Paso 3 ($k=3$):** El mínimo en Q es $u=\mathrm{D}$ ($d[\mathrm{D}]=400$). $S=\{\mathrm{A},\mathrm{B},\mathrm{D}\}$. Relajaciones:
  - (D,E): $400+250=650<d[\mathrm{E}](\infty) \implies d[\mathrm{E}]\leftarrow 650$, $\pi[\mathrm{E}]\leftarrow \mathrm{D}$, `Decrease-Key`$(Q,\mathrm{E},650)$.
  - (D,C): $400+500=900\not< d[\mathrm{C}](450)$. Se descarta.
- **Paso 4 ($k=4$):** Se extrae $u=\mathrm{C}$ ($d[\mathrm{C}]=450$). $S=\{\mathrm{A},\mathrm{B},\mathrm{D},\mathrm{C}\}$. Relajaciones:
  - (C,F): $450+350=800<d[\mathrm{F}](\infty) \implies d[\mathrm{F}]\leftarrow 800$, $\pi[\mathrm{F}]\leftarrow \mathrm{C}$, `Decrease-Key`$(Q,\mathrm{F},800)$.
  - (C,E): $450+600=1050\not< d[\mathrm{E}](650)$. Se descarta.
- **Paso 5 ($k=5$):** Se extrae $u=\mathrm{E}$ ($d[\mathrm{E}]=650$). $S=\{\mathrm{A},\mathrm{B},\mathrm{D},\mathrm{C},\mathrm{E}\}$. Relajaciones:
  - (E,F): $650+700=1350\not< d[\mathrm{F}](800)$. Se descarta.
- **Paso 6 ($k=6$):** Se extrae el último vértice $u=\mathrm{F}$ ($d[\mathrm{F}]=800$). $S=V$. La cola Q queda vacía. Finaliza la ejecución.

Tabla 6

Traza matricial de distancias mínimas y predecesores en cada iteración del algoritmo.

Iter.	**Vértice u**	**Acción / Relajación**	**A**	**B**	**C**	**D**	**E**	**F**
$k=0$	Inicial	Inserción en cola	0 / -	∞ / -	∞ / -	∞ / -	∞ / -	∞ / -
$k=1$	A ($d=0$)	Relaja B (180), Relaja C (450)	**0** / -	180 / A	450 / A	∞ / -	∞ / -	∞ / -
$k=2$	B ($d=180$)	Relaja D (400), Descarta C	**0** / -	**180** / A	450 / A	400 / B	∞ / -	∞ / -
$k=3$	D ($d=400$)	Relaja E (650), Descarta C	**0** / -	**180** / A	450 / A	**400** / B	650 / D	∞ / -
$k=4$	C ($d=450$)	Relaja F (800), Descarta E	**0** / -	**180** / A	**450** / A	**400** / B	650 / D	800 / C
$k=5$	E ($d=650$)	Descarta F	**0** / -	**180** / A	**450** / A	**400** / B	**650** / D	800 / C
$k=6$	F ($d=800$)	Sin aristas salientes	**0** / -	**180** / A	**450** / A	**400** / B	**650** / D	**800** / C

Nota. Notación *distancia / predecesor* ($d[v]/\pi[v]$). Cifras en negrita indican nodos cerrados en S.

A partir del vector de predecesores $\pi=\{\mathrm{A}:\mathrm{NIL},\mathrm{B}:\mathrm{A},\mathrm{C}:\mathrm{A},\mathrm{D}:\mathrm{B},\mathrm{E}:\mathrm{D},\mathrm{F}:\mathrm{C}\}$, se reconstruyen unívocamente las rutas mínimas desde la Plaza Mayor (A):

$$\text{Ruta hacia Mercado San Pedro (E):}\quad \mathrm{A}\to\mathrm{B}\to\mathrm{D}\to\mathrm{E}\quad (180+220+250=650.0\ \mathrm{m}), \tag{17}$$

$$\text{Ruta hacia Plazoleta Limacpampa (F):}\quad \mathrm{A}\to\mathrm{C}\to\mathrm{F}\quad (450+350=800.0\ \mathrm{m}). \tag{18}$$

Bajo la métrica de tiempo efectivo con impedancia topográfica (Tabla 5), los costos son 165.33 s y 144.45 s, respectivamente, coincidiendo exactamente con la salida del software (Tabla 7).

Tabla 7
Resultados computacionales de distancias y tiempos óptimos calculados desde Plaza Mayor

Nodo	Nombre de la Intersección	Distancia Pura (m)	Tiempo Efectivo (s)	Ruta óptima
A	Plaza Mayor del Cusco	0.0	0.0	`A`
B	Plaza Regocijo (Cusipata)	180.0	44.8	`A ->B`
C	Qorikancha (Santo Domingo)	450.0	108.0	`A ->C`
D	Plaza San Francisco	400.0	101.6	`A ->B ->D`
E	Mercado Central San Pedro	650.0	165.3	`A ->B ->D ->E`
F	Plazoleta Limacpampa	800.0	144.4	`A ->C ->F`

Nota. Solución calculada desde el vértice fuente $s =$ A (Plaza Mayor).

5.7. Instancia de Verificación Canónica: CLRS Figura 24.6

Para comprobar la adhesión a la literatura clásica de algoritmos, el sistema incluye como instancia canónica el grafo dirigido de 5 nodos de Cormen et al. (2022, p. 659). El orden de extracción obtenido por el software es estrictamente $\langle s, y, z, t, x \rangle$ con distancias definitivas $\langle 0, 5, 7, 8, 9 \rangle$, idénticas a la solución teórica del libro de texto.

5.8. Red Vial Urbana Real de Cusco (OpenStreetMap)

La red metropolitana real fue extraída mediante OSMnx 2.1.1 en torno al centro urbano de Cusco $(-13.5165, -71.9788)$ con un radio de 1,500 metros en modalidad `drive`. La topología serializada en `cusco_osm_drive.json` contiene 891 intersecciones y 2,139 tramos viales dirigidos, arrojando una densidad de $|E|/|V| = 2.40$ característica de mallas viales dispersas. Sobre esta red, la consulta canónica entre el centroide y el nodo alcanzable más lejano abarca 17 intersecciones, 2,810.6 metros y un tiempo base óptimo de 387.197 segundos.

6. Diseño Algorítmico

6.1. Algoritmo de Dijkstra con Cola de Prioridad Abstracta

El Algoritmo de Dijkstra es el método canónico para la resolución del problema de caminos mínimos desde un origen único en grafos con pesos no negativos. Su estructura lógica opera sobre una cola de prioridad genérica Q. El procedimiento detallado se formaliza en el Algoritmo 1.

Algorithm 1 Algoritmo de Dijkstra con cola de prioridad parametrizada

Require: Grafo dirigido $G = (V, E, w)$, Vértice origen $s \in V$, Cola de prioridad parametrizada Q
Ensure: Vector de distancias mínimas $d : V \to \mathbb{R}_{\geq 0} \cup \{\infty\}$, Predecesores $\pi : V \to V \cup \{\text{NIL}\}$

// [L1-L6] Inicialización del estado algorítmico
for all $v \in V$ **do**
  $d[v] \leftarrow \infty$
  $\pi[v] \leftarrow$ NIL
end for
$d[s] \leftarrow 0.0$
Q.insert$(s, 0.0)$ {O inserción global de V con claves iniciales}
$S \leftarrow \emptyset$ {Conjunto de vértices con distancia permanente cerrada}
while not Q.is_empty() **do**
  $(d_u, u) \leftarrow Q$.extract_min() {// [L9-L10] Extracción del vértice con menor distancia tentativa}
  if $d_u = \infty$ **then**
    break {// [L11-L13] Vértices restantes inalcanzables desde s}
  end if
  $S \leftarrow S \cup \{u\}$ {// [L14] Incorporación de u al conjunto permanente}
  for all $(u, v) \in E$ **do**
    if $v \notin S$ **and** $d_u + w(u, v) < d[v]$ **then**
      $d[v] \leftarrow d_u + w(u, v)$ {// [L15-L18] Relajación voraz de la arista}
      $\pi[v] \leftarrow u$
      if $v \in Q$ **then**
        Q.decrease_key$(v, d[v])$ {// [L19-L21] Disminución de prioridad en Q}
      else
        Q.insert$(v, d[v])$
      end if
    end if
  end for
end while
return (d, π)

Correspondencia entre pseudocódigo y código fuente. Para asegurar la trazabilidad estricta entre la literatura algorítmica y el paquete ejecutable, cada bloque relevante en `python/src/algorithms/dijkstra.py` implementa fielmente las líneas del Algoritmo 1, como se sintetiza en la Tabla 8.

Tabla 8
Correspondencia entre pseudocódigo y la implementación en Python

Etapa Algorítmica	**Pseudocódigo [Lk]**	**Código Python** (`dijkstra.py`)
Inicialización	$d[v] \leftarrow \infty$, $\pi[v] \leftarrow$ NIL, $d[s] \leftarrow 0$	Diccionarios `dist`, `prev` y asignación `dist[s]=0.0`.
Construcción de Q	Q.insert$(s,0)$ o Q.build(V,d)	Factoría `create_queue(queue_type)` e inserción controlada.
Extracción del mínimo	$(d_u,u) \leftarrow Q$.extract_min()	Invocación polimórfica `d_u, u = heap.extract_min()`.
Criterio de corte	**if** $d_u = \infty$ **then break**	`if d_u == INF: break` (componentes disjuntas).
Relajación voraz	$d_u + w(u,v) < d[v]$	Comprobación `candidate = d_u + weight <dist[v]`.
Actualización de cola	`decrease_key` o `insert`	`heap.decrease_key(v, dist[v])` si está en cola; si no, `insert`.
Reconstrucción	Retorno de (d,π) y traza	Módulo `reconstruct_path(prev, s, t)` y telemetría de operaciones.

6.2. Invariantes de Bucle y Demostración Formal de Corrección

La corrección formal del algoritmo descansa sobre el siguiente invariante inductivo:

Invariante de bucle. Al inicio de cada iteración del ciclo `while` (línea 8):

1. Para todo vértice permanente $u \in S$, el valor estimado coincide con la distancia mínima real: $d[u] = \delta(s,u)$.
2. Para todo vértice no permanente $v \in Q = V \setminus S$, $d[v]$ es la longitud del camino más corto desde s hasta v que transita exclusivamente por vértices pertenecientes al conjunto S.

Demostración por inducción matemática:

- **Base ($|S| = 0$):** Antes de la primera iteración, $S = \emptyset$. Para el origen, $d[s] = 0 = \delta(s,s)$ dado que no existen aristas con pesos negativos ($w(e) \geq 0$). Para todo $v \neq s$, $d[v] = \infty$, lo cual satisface la cota superior sobre caminos sin vértices intermedios en S.
- **Paso inductivo:** Supóngase que el invariante se satisface para $|S| = k$. El algoritmo selecciona el vértice $u \in Q$ con $d_u = \min_{z \in Q} d[z]$ y lo incorpora a S. Si existiese un camino dirigido P desde s hasta u estrictamente más corto que $d[u]$ ($w(P) < d[u]$), dicho camino debe salir de S hacia Q por primera vez a través de alguna arista (x,y), con $x \in S$ y $y \in Q$. La longitud acumulada hasta y satisface:

$$w(P) \geq \delta(s,x) + w(x,y) = d[y] \tag{19}$$

  dado que $x \in S$ cumple la hipótesis inductiva y (x,y) ya fue relajada al incorporar x. Puesto que u minimiza Q y los costos son no negativos ($w \geq 0$), se tiene $d[y] \geq d[u]$, lo cual implica $w(P) \geq d[u]$, contradiciendo la existencia de un camino más corto. Por ende, $d[u] = \delta(s,u)$. Tras relajar las aristas salientes de u, la segunda propiedad del invariante se restaura para la siguiente iteración.

- **Terminación:** Cuando $Q = \emptyset$ o los vértices restantes tienen distancia ∞, S contiene todos los nodos alcanzables desde s, garantizando que $d[v] = \delta(s,v)$ y que el árbol de predecesores π codifica las rutas óptimas.

6.3. Estructuras de Datos de Cola de Prioridad Evaluadas

Min-Heap Binario Indexado (`IndexedMinHeap`)

El montículo binario indexado se representa sobre un arreglo contiguo en memoria. Para posibilitar la operación `decrease_key` en tiempo sublineal sin requerir búsquedas lineales exhaustivas, se implementa una tabla hash inversa de posiciones $\text{pos} : V \to \mathbb{N}$. Cada movimiento de elementos durante el flotado (`sift-up`) o hundido (`sift-down`) ejecuta un intercambio atómico:

$$\text{heap}[i] \leftrightarrow \text{heap}[j], \quad \text{pos}[\text{heap}[i].\text{id}] \leftarrow i, \quad \text{pos}[\text{heap}[j].\text{id}] \leftarrow j \tag{20}$$

Con este esquema, localizar cualquier vértice toma $O(1)$ y el flotado recorre a lo sumo $\lfloor \log_2 |V| \rfloor$ niveles, logrando una complejidad estricta de $O(\log |V|)$ por decremento con un consumo de memoria lineal $O(|V|)$.

4-ary Heap Contiguo (`DAryHeap`, $d = 4$)

El montículo 4-ario generaliza el heap binario adoptando un factor de ramificación $d = 4$. La navegación entre niveles se realiza mediante aritmética entera directa sin punteros:

$$\text{padre}(i) = \left\lfloor \frac{i-1}{4} \right\rfloor, \quad \text{hijo}_k(i) = 4i + k, \quad k \in \{1,2,3,4\} \tag{21}$$

Esta formulación reduce la profundidad del árbol a la mitad: $\lceil \log_4 |V| \rceil = 0.5 \lceil \log_2 |V| \rceil$. En `decrease_key`, el número de intercambios se reduce a la mitad. Durante `extract_min`, el método `sift-down` debe identificar el mínimo entre cuatro descendientes contiguos en memoria, lo cual encaja perfectamente en una única línea de caché L1/L2 de 64 bytes de la CPU contemporánea (LaMarca & Ladner, 1996).

Fibonacci Heap (`FibonacciHeap`)

El Fibonacci Heap (Fredman & Tarjan, 1987) mantiene una colección de árboles con orden de montículo estructurados mediante listas circulares doblemente enlazadas con punteros `parent`, `child`, `left`, `right`, `degree` y el bit de marcado `marked`.

- `insert`: Inserta un nuevo árbol unitario en la lista de raíces en $O(1)$ peor caso.
- `decrease_key`: Si la clave relajada viola el orden de montículo con su padre, se corta el nodo hacia la lista de raíces en $O(1)$ amortizado. Si el padre ya estaba marcado, se propaga un corte en cascada recursivo (*cascading cut*) garantizando que el grado de cualquier nodo satisfaga $D(n) \leq \lfloor \log_\phi n \rfloor \approx 1.4404 \log_2 n$.

- `extract_min`: Elimina el nodo raíz mínimo, promueve todos sus hijos a la lista de raíces y ejecuta una consolidación sistemática uniendo árboles de igual grado mediante un arreglo dinámico dimensionado con la cota de la razón áurea $\lceil \log_{\phi} n \rceil + 2$.

6.4. Análisis Comparativo de Complejidades Teóricas

La Tabla 9 expone la comparación de cotas asintóticas temporales y espaciales entre las tres estructuras analizadas y la referencia elemental de arreglo no ordenado.

Tabla 9
Cotas temporales y espaciales de las colas de prioridad evaluadas

Estructura	Insert	Extract-Min	Decrease-Key	Espacio	Costo total en Dijkstra
Arreglo no ordenado	$O(1)$	$O(\|V\|)$	$O(1)$	$O(\|V\|)$	$O(\|V\|^2 + \|E\|)$
Heap binario	$O(\log \|V\|)$	$O(\log \|V\|)$	$O(\log \|V\|)$	$O(\|V\|)$	$O((\|V\| + \|E\|) \log \|V\|)$
Heap 4-ario ($d = 4$)	$O(\log_4 \|V\|)$	$O(4 \log_4 \|V\|)$	$O(\log_4 \|V\|)$	$O(\|V\|)$	$O((\|E\| + 4\|V\|) \log_4 \|V\|)$
Fibonacci Heap	$O(1)$	$O(\log \|V\|)^*$	$O(1)^*$	$O(\|V\|)$	$O(\|E\| + \|V\| \log \|V\|)$

Nota. *Cota amortizada. Para d fijo, $\log_d |V| = O(\log |V|)$; las discrepancias empíricas provienen de las constantes de indirección y la jerarquía de memoria física.

6.5. Guía Práctica de Selección según el Escenario

La Tabla 10 sintetiza el marco de selección tecnológica deducido a partir de los fundamentos algorítmicos y el perfil de la red vial.

Tabla 10
Guía de selección práctica de algoritmos y estructuras en redes viales

Alternativa	Escenario Recomendado	Justificación Técnica y Evidencia
Dijkstra + Heap Binario	Redes viales pequeñas y medianas ($\|V\| \leq 1,000$), donde se prioriza simplicidad y mínima sobrecarga.	Menor constante de inicialización; obtuvo el menor tiempo en la red real OSM de Cusco (15.65 ms) y en 3 de 10 escalas sintéticas.
Dijkstra + Heap 4-ario	Redes viales metropolitanas extensas ($\|V\| \geq 1,000$) sobre procesadores con líneas de caché de 64 bytes.	Menor mediana en 7 de 10 escalas sintéticas (ventaja mediana de 4–5 %); aprovecha la reducción de niveles y la contigüidad espacial.
Dijkstra + Fibonacci	Grafos hiper-densos ($\|E\| \gg \|V\| \log \|V\|$) o entornos de bajo costo de asignación en memoria (C puro).	En Python no resultó competitivo (1.12–1.59 veces más lento); la constante de punteros domina frente al ahorro en comparaciones.
A^* **con heurística consistente**	Consultas punto a punto ($s \to t$) para navegación y vehículos de emergencia con coordenadas WGS84.	Reduce las expansiones de nodos en un 7.2 % en la red OSM sin degradar la optimalidad de la ruta.
Bellman-Ford	Oráculo de validación cruzada y verificación de ausencia de ciclos negativos.	Complejidad $O(\|V\|\|E\|)$; opera como juez independiente para validar la suite de pruebas unitarias.

7. Diseño e Implementación del Sistema

7.1. Arquitectura Modular del Sistema (4 Capas)

El sistema computacional ha sido concebido bajo una arquitectura desacoplada en cuatro capas con dependencias estrictamente unidireccionales y descendentes (Figura 2). El núcleo de cálculo de caminos mínimos es completamente independiente de la interfaz gráfica web, de los mecanismos de almacenamiento en disco y de la estructura concreta de cola de prioridad empleada. La inyección abstracta de colas permite evaluar y sustituir componentes sin modificar el algoritmo de Dijkstra ni reescribir los módulos de ruteo.

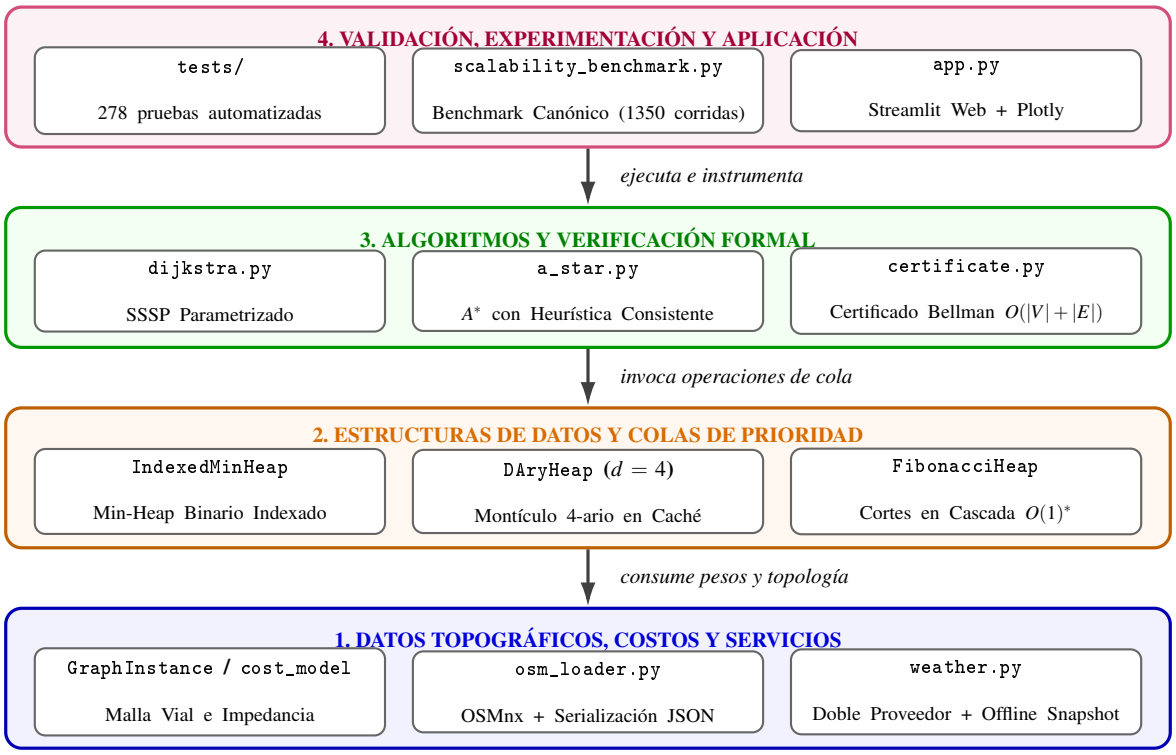


Figura 2
Arquitectura modular desacoplada del sistema en cuatro capas operativas

7.2. Descripción Detallada de Módulos y Componentes

El software se estructura modularmente dentro del directorio `python/`:

1. `src/algorithms/dijkstra.py`: Motor algorítmico principal que implementa el algoritmo de Dijkstra con inyección de colas de prioridad y telemetría de operaciones elementales.

2. `src/algorithms/a_star.py`: Búsqueda heurística A^* con precomputación del factor admisible λ, construcción de heurística consistente euclidiana escalada y métricas de poda.

3. `src/algorithms/reference.py`: Oráculos de referencia independientes (Bellman-Ford y Dijkstra ingenuo sobre lista no ordenada).

4. `src/datastructures/`: Implementación pura de `IndexedMinHeap`, `DAryHeap` ($d = 4$) y `FibonacciHeap` bajo contrato polimórfico común.
5. `src/io/graph_io.py`: Ingestión, parsing y validación de grafos en JSON, listas de adyacencia y formato de red vial andina.
6. `src/io/osm_loader.py`: Ingestión, reproyección y serialización de redes OpenStreetMap mediante OSMnx, con soporte nativo de *snapshots* desconectados.
7. `src/services/weather.py`: Servicio meteorológico desacoplado con arquitectura de doble proveedor (Open-Meteo REST API y OpenWeatherMap API v2.5), persistencia en *snapshots* y respaldo ante fallos de conexión.
8. `src/services/traffic.py`: Modelo de congestión vehicular horaria y coeficientes por clase de vía.
9. `src/verification/certificate.py`: Validador del certificado formal de optimalidad en $O(|V|+|E|)$ y clasificador de equivalencia de soluciones.
10. `src/benchmarks/scalability_benchmark.py`: Banco de experimentación sistemática para medir tiempos y primitivas en perfiles sintéticos dispersos y densos.
11. `app.py`: Aplicación web interactiva desarrollada en Streamlit con visualizaciones geográficas Plotly y cinco módulos operativos.
12. `main.py` y `src/cli.py`: Interfaz de línea de comandos para automatización y demostración en vivo.

7.3. Formatos y Estructura de Datos (Serialización JSON)

Todos los artefactos de datos se serializan en formato JSON estructurado y versionado, garantizando que las pruebas no dependan de APIs externas en vivo (Tabla 11).

Tabla 11
Archivos de datos principales y su rol funcional en el sistema

Archivo en Repositorio	**Contenido y Propósito**
`data/instances/cusco_centro.json`	Instancia nuclear didáctica (6 nodos, 9 aristas, longitudes, pendientes y tiempos).
`data/processed/cusco_osm_drive.json`	Snapshot oficial de Cusco OSM (891 nodos con lat/lon WGS84, 2,139 aristas dirigidas).
`data/processed/benchmark_results.json`	Protocolo experimental, ambiente, muestras cronometradas, medianas e IQR de las 10 escalas.
`data/processed/applied_results.json`	Mediciones de la consulta OSM, comparación de colas, poda de A^* y sensibilidad climática.
`data/processed/weather_cusco.json`	Snapshot meteorológico local actualizado de contingencia.

```json
{
  "origin": "A",
  "destination": "B",
  "length_m": 180.0,
  "street_type": "calle_centro",
  "gradient_pct": 1.5,
  "street_name": "Calle Espaderos / Portal Carnicerias",
  "travel_time_sec": 44.82
}
```

Listing 1: Extracto del esquema JSON serializado para un segmento vial cusqueño

7.4. Contratos de Interfaz e Inyección de Colas de Prioridad

Para posibilitar la sustitución dinámica y transparente de colas en el solver de Dijkstra sin alterar el flujo algorítmico, cada una de las tres estructuras implementa una interfaz uniforme:

- `insert(item: str, key: float) ->None`: Inserta un identificador de vértice con su distancia tentativa asociada.
- `extract_min() ->Tuple[float, str]`: Extrae y retorna el par $(\text{clave}, \text{elemento})$ de menor valor acumulado, reestructurando el montículo según sus invariantes.
- `decrease_key(item: str, new_key: float) ->None`: Disminuye la clave del vértice a su nuevo valor relajado, flotando el nodo o ejecutando cortes en cascada.
- `is_empty() ->bool`: Determina en $O(1)$ si existen vértices pendientes de procesamiento.
- `op_stats() ->Dict[str, int]`: Retorna la telemetría acumulada de inserciones, extracciones, decrementos de clave y comparaciones.

7.5. Manejo Defensivo de Entradas y Validación

La validación centralizada en `src/io/graph_io.py` protege al sistema contra datos corruptos, rechazando de forma determinista: JSON mal formado, campos requeridos faltantes, identificadores duplicados, pesos no numéricos, NaN o infinitos, costos negativos (en Dijkstra) y nodos de origen/destino inexistentes. La CLI retorna códigos de salida estandarizados: 0 (éxito), 1 (fallo en la verificación del certificado de optimalidad) y 2 (entrada inválida o error de sintaxis).

7.6. Servicios Complementarios: Meteorología y Escenarios de Tráfico

El módulo `src/services/weather.py` implementa un desacoplamiento mediante una arquitectura de doble proveedor:

1. **Open-Meteo REST API (Open-Meteo, 2026):** Fuente pública libre sin autenticación basada en datos de modelos meteorológicos de predicción numérica.

2. **OpenWeatherMap API (v2.5 Current Weather) (OpenWeather, 2026):** Proveedor autenticado mediante clave de API suministrada exclusivamente por variables de entorno locales, con mapeo hacia `WeatherObservation`.

Ambos clientes alimentan la función multiplicadora $\Omega_{\text{clima}} \geq 1.0$:

$$\Omega_{\text{clima}} = 1.0 + c_{\text{lluvia}}\,\text{mín}\left(\frac{p}{10}, 1.0\right) + c_{\text{vis}}\,\text{máx}\left(0, 1 - \frac{v}{10\,000}\right) + c_{\text{ráfaga}}\,\text{mín}\left(\frac{r}{80}, 1.0\right) + c_{\text{hum}} f(\text{HR}) \tag{22}$$

Si la conectividad a Internet falla o la clave presenta latencia de activación, el sistema opera sin interrupciones consumiendo los datos locales del *snapshot*.

7.7. Decisiones de Implementación Relevantes y Justificación Técnica

- **Programación pura sin `heapq`:** Las tres colas se programaron desde cero en Python puro para garantizar que las mediciones reflejen las propiedades estructurales de los algoritmos y no optimizaciones escritas en lenguaje C.
- **Degeneración admisible de A^*:** Si un grafo no posee coordenadas geográficas, A^* adopta $h(v) = 0$, degenerando de manera admisible en el algoritmo canónico de Dijkstra sin romper la ejecución.
- **Reproducibilidad sin dependencias en vivo:** La red vial urbana y los datos climáticos se congelan en archivos serializados en `data/processed/`, garantizando ejecuciones deterministas en cualquier computadora sin conexión a red.
- **Separación estricta entre núcleo y visualización:** La capa matemática y algorítmica es 100 % agnóstica de Streamlit, asegurando que la suite de pruebas unitarias y los benchmarks se ejecuten sin sobrecargas de renderizado web.

8. Estrategia de Pruebas

8.1. Framework de Pruebas y Patrón Arrange-Act-Assert (AAA)

Para garantizar la confiabilidad, correctitud matemática y robustez del software, el proyecto descansa sobre una suite automatizada de pruebas implementada con el framework `pytest`. La totalidad de las 278 pruebas se estructuran formalmente bajo el patrón **Arrange - Act - Assert (AAA)**:

- **Arrange (Organizar):** Configuración del entorno determinista, definición de semillas pseudoaleatorias fijas, instanciación del grafo de prueba o cola concreta y declaración explícita de los valores esperados.
- **Act (Actuar):** Invocación del método o solver bajo evaluación (`dijkstra_solver`, `a_star_solver`, operaciones primitivas de cola o comandos de la CLI).
- **Assert (Afirmar):** Comprobación determinista mediante aserciones de igualdad de distancias con tolerancia flotante $\varepsilon = 10^{-6}$, preservación de invariantes, captura de excepciones y códigos de salida.

```
python -m pytest python/tests -q
```

Listing 2: Comando de ejecución de la suite automatizada completa

8.2. Tipología y Cobertura de la Suite Automatizada (278 Pruebas)

La suite reúne 278 pruebas automatizadas que cubren exhaustivamente las cuatro capas de la arquitectura (Tablas 12 y 13).

Tabla 12
Cobertura de las pruebas automatizadas por categoría funcional

Categoría	Módulos de Prueba	Casos y Propiedades Evaluadas
Pruebas unitarias	`test_priority_queues.py`, `test_dijkstra.py`	Invariantes de montículo, `sift-up`, `sift-down`, cortes en cascada, consolidación dinámica y contratos abstractos.
Pruebas de integración	`test_cli.py`, `test_osm_loader.py`, `test_weather.py`, `test_streamlit_app.py`	Pipeline CLI, ingesta OSMnx, respaldo offline de clima, interfaz Streamlit y visualización vectorial Plotly.
Validación cruzada	`test_equivalence.py`, `test_validation_io.py`	Comparación contra Bellman-Ford, Dijkstra ingenuo, NetworkX y certificado de optimalidad.
Casos límite	`test_dijkstra_manual.py`, `test_dijkstra.py`	Grafo unitario, componentes disjuntas, lazos, aristas paralelas, costos cero y pesos extremos.
Casos adversos	`test_validation_io.py`, `test_priority_queues.py`, `test_a_star.py`	Pesos negativos, ciclos negativos, archivos corruptos, claves no numéricas y heurísticas inadmisibles.

Tabla 13
Distribución modular de la suite de 278 pruebas

Archivo de Prueba	Cantidad	Propósito y Aserciones
`test_priority_queues.py`	61	Contrato polimórfico común, orden estricto de extracción, `decrease_key` y estrés con `heapq`.
`test_equivalence.py`	48	Equivalencia matemática de distancias con Bellman-Ford, NetworkX y Dijkstra en 40 grafos.
`test_validation_io.py`	45	Detección de JSON corruptos, entradas adversas y validación del certificado formal.
`test_dijkstra_manual.py`	36	Traza analítica de Cusco, instancia CLRS 24.6, nodos aislados y lazos.
`test_cli.py`	25	Comandos de consola, códigos de salida (0, 1, 2) y ciclo generación–exportación.
`test_dijkstra.py`	11	Casos base de SSSP, límites y equivalencia a escala.
`test_a_star.py`	8	Poda de búsqueda, admisibilidad, consistencia y fallback cuando no hay coordenadas.
`test_weather.py`	12	Servicio climático, respuestas simuladas, manejo de 401 y modo desconectado.
`test_streamlit_app.py`	11	Integración de Streamlit sin red, renderizado de mapas y protección de credenciales.
`test_traffic.py`	10	Escenarios horarios y factores de congestión vial.
`test_osm_loader.py`	4	Carga determinista de OpenStreetMap y serialización JSON.
`test_benchmark_protocol.py`	4	Estadísticas robustas (mediana e IQR) y estructura del benchmark.
`test_applied_analysis.py`	3	Consulta sobre la red real de Cusco y sensibilidad ambiental.
Total	**278**	Suite completa de pruebas automatizadas aprobadas.

8.3. Metodología de Verificación y Oráculos Independientes

Para evitar sesgos algorítmicos derivados de errores compartidos entre implementaciones del mismo equipo, la exactitud de las soluciones se verifica contra tres oráculos independientes:

1. **Algoritmo de Bellman-Ford:** Implementación pura de $O(|V||E|)$ tolerante a costos negativos y capaz de diagnosticar ciclos negativos. Se utiliza en 40 grafos aleatorios con topologías y densidades diversas.

2. **Dijkstra sobre Arreglo no Ordenado:** Algoritmo determinista $O(|V|^2 + |E|)$ que no emplea estructuras de árbol ni montículos, confirmando que la lógica de prioridades no perturba las distancias.

3. **Librería Canónica NetworkX 3.7:** Empleada exclusivamente como oráculo externo de referencia (*ground truth*) sobre mallas cuadrículas dirigidas de 8×8.

4. **Instancias Resueltas Analíticamente a Mano:** Contraste contra la traza de 6 nodos de Cusco (Sección 5) y la Figura 24.6 de Cormen et al. (2022).

8.4. Certificado Formal de Optimalidad de Bellman $O(|V|+|E|)$

Para validar la optimalidad sin reejecutar un solver completo, se implementó un certificador lineal que comprueba cuatro condiciones matemáticas necesarias y suficientes:

1. **Distancia al origen:** $d[s] = 0.0$.
2. **Ausencia de relajaciones pendientes:** Para toda arista dirigida $(u,v) \in E$ con $d[u] < \infty$, se satisface estrictamente:
$$d[v] \leq d[u] + w(u,v) \tag{23}$$
garantizando que ninguna distancia tentativa esté sobreestimada.
3. **Conexión estricta de predecesores al origen:** Para todo vértice alcanzable $v \neq s$, existe una cadena finita acíclica de aristas ajustadas ($d[v] = d[\pi[v]] + w(\pi[v], v)$) que converge al origen s, detectando y rechazando ciclos desconectados de costo cero.
4. **Consistencia de trayectorias:** Cada camino explícito reconstruido conecta s con v, utiliza únicamente tramos existentes en E y su costo sumado coincide de forma exacta con $d[v]$.

La verificación de las desigualdades y el árbol de predecesores opera en tiempo estricto $O(|V|+|E|)$.

8.5. Casos Límite, Condiciones Adversas y Pruebas de Regresión

La suite de pruebas incluye escenarios extremos diseñados para someter al sistema a condiciones no nominales:

- **Casos límite:** Grafos unitarios ($|V| = 1, |E| = 0$), orígenes aislados sin aristas salientes, componentes disjuntas inalcanzables ($d[v] = \infty$), lazos dirigidos ((u,u)), aristas paralelas múltiples con distintos costos, aristas de costo cero y aristas con pesos extremos (10^{12} y 10^{-9}).
- **Casos adversos:** Entradas con pesos negativos, grafos con ciclos negativos (detectados por Bellman-Ford y rechazados por Dijkstra), JSONs con sintaxis rota, claves NaN, cadenas de texto o nulas, e intentos de incremento de clave en `decrease_key`.
- **Prueba de regresión de A^*:** Detección y rechazo de heurísticas inadmisibles producidas por coordenadas geográficas inconsistentes o proyecciones distorsionadas, forzando la degradación segura hacia Dijkstra ($h = 0$).

8.6. Pruebas de Integración y Funcionamiento Desconectado

Para asegurar que el sistema sea completamente reproducible y resiliente en ambientes sin conexión a Internet, 11 pruebas de integración verifican: la ingesta de *snapshots* locales de OpenStreetMap sin descargas en vivo, el aislamiento de llamadas meteorológicas mediante respuestas locales simuladas (`mocking`), la renderización de mapas vectoriales en Streamlit mediante `AppTest` y la protección de claves de API en memoria sin persistencia en archivos versionados.

8.7. Criterios de Aceptación y Calidad del Software

Una versión del prototipo se califica como aprobada únicamente cuando: (1) Las 278 pruebas unitarias e integrales finalizan con estado `PASSED`; (2) El certificado formal de optimalidad se satisface al 100 % en todas las corridas experimentales; (3) Las tres colas de prioridad producen distancias matemáticamente indistinguibles ($\varepsilon < 10^{-6}$); y (4) Tanto la CLI como la plataforma web Streamlit son totalmente operativas en modo desconectado.

9. Metodología Experimental

9.1. Preguntas de Investigación e Hipótesis de Evaluación

El diseño experimental responde a tres interrogantes computacionales centrales:

1. **PI1:** ¿Qué estructura de cola de prioridad minimiza el tiempo de pared (*wall-clock time*) en el Algoritmo de Dijkstra sobre redes viales sintéticas dispersas y densas, y cómo evoluciona dicha relación frente al incremento de escala?
2. **PI2:** ¿Se traduce la cota asintótica teórica $O(1)$ amortizada en `Decrease-Key` del Fibonacci Heap en un menor tiempo real de ejecución en un entorno de software interpretado en Python 3.12?
3. **PI3:** ¿Se reproduce el comportamiento observado en el banco sintético sobre la red vial urbana real de Cusco (OpenStreetMap), y cuál es la ganancia cuantitativa de la extensión heurística A^* y el análisis de sensibilidad climática?

Hipótesis de trabajo:

- **H1 (Localidad de memoria en redes dispersas):** En redes viales dispersas ($|E| \approx 4|V|$), los montículos contiguos basados en arreglos (`IndexedMinHeap` y `DAryHeap`) superarán en tiempo de pared al `FibonacciHeap`, debido a la menor sobrecarga de asignación dinámica y a la explotación de líneas de caché de CPU.
- **H2 (Efecto de la densidad y factor de ramificación):** Al incrementarse la densidad del grafo ($|E| \propto |V|^2$), el mayor volumen de operaciones `Decrease-Key` favorecerá relativamente al montículo 4-ario frente al binario gracias a la reducción en un 50 % de la profundidad del árbol.
- **H3 (Poda espacial con búsqueda heurística):** El algoritmo A^* con heurística euclidiana escalada consistente reducirá el número de expansiones de vértices respecto a Dijkstra sin alterar el costo óptimo de la ruta.
- **H4 (Invariancia topológica ante factores uniformes):** La aplicación de un factor meteorológico o de congestión uniforme incrementará el tiempo de viaje estimado, pero preservará exactamente la secuencia de aristas que componen la ruta mínima óptima.

9.2. Diseño Factorial e Instancias Experimentales

El banco canónico de evaluación implementa un diseño factorial completo $3 \times 2 \times 5$:

- **Estructura de cola de prioridad (3 niveles):** Min-Heap Binario Indexado, 4-ary Heap ($d = 4$) y Fibonacci Heap.
- **Perfil de densidad del grafo (2 niveles):**
  - *Perfil disperso:* $|E| = 4|V|$, representativo de la baja densidad planar de redes viales metropolitanas.

- *Perfil denso:* $|E| = 0.08\,|V|(|V|-1)$, correspondiente al 8 % de conectividad cuadrática total.

- **Escala del grafo (5 niveles por perfil):**
  - Perfil disperso: $|V| \in \{100, 500, 1000, 2500, 5000\}$.
  - Perfil denso: $|V| \in \{50, 100, 200, 400, 800\}$.

Cada celda experimental evalúa 3 instancias pseudoaleatorias generadas con semilla fija determinista, y sobre cada instancia se registran 15 réplicas temporales independientes precedidas por 2 corridas de calentamiento (*warm-up*):

$$2 \text{ perfiles} \times 5 \text{ escalas} \times 3 \text{ instancias} \times 3 \text{ colas} \times 15 \text{ réplicas} = 1,350 \text{ muestras cronometradas} \tag{24}$$

Generación determinista de instancias sintéticas. Las redes se construyen mediante el módulo `benchmark_network`. Para garantizar conectividad y descartar componentes triviales desconectadas, el generador implanta primero una cadena dirigida hamiltoniana $N_0 \to N_1 \to \cdots \to N_{n-1}$ (asegurando que todos los nodos sean alcanzables desde el origen), incorporando posteriormente aristas dirigidas aleatorias con pesos continuos uniformes en el rango $[1.0, 1000.0]$ metros hasta colmar la cuota $|E|$. Las semillas están estrictamente parametrizadas por escala n e índice $j \in \{0, 1, 2\}$:

$$s(n, j) = \begin{cases} 12345 + 31n + j, & \text{para perfil disperso,} \\ 22345 + 31n + j, & \text{para perfil denso.} \end{cases} \tag{25}$$

9.3. Variables Experimentales y Métricas de Desempeño

- **Variables independientes:** Estructura de cola, escala de vértices $|V|$, densidad de aristas $|E|$ y tipo de grafo (sintético controlado vs. OpenStreetMap real).

- **Variables dependientes:**
  - Tiempo de pared neto (*wall-clock time*) registrado en milisegundos (ms).
  - Estadísticos de tendencia central y dispersión: Mediana e Intervalo Intercuartílico (IQR), seleccionados por su robustez frente a distribuciones asimétricas y ruido del sistema operativo.
  - Conteo unívoco de primitivas algorítmicas: inserciones (`Insert`), extracciones (`Extract-Min`), decrementos efectivos de prioridad (`Decrease-Key`), intentos totales de relajación y comparaciones de clave ejecutadas.

9.4. Entorno Experimental Controlado e Instrumentación de Precisión

Todas las mediciones se condujeron sobre una plataforma computacional homogénea e instrumentada para minimizar perturbaciones externas (Tabla 14).

Tabla 14
Especificaciones del entorno experimental de medición

Componente / Parámetro	**Especificación Técnica**
Sistema Operativo	Microsoft Windows 11 Pro 64-bit (Build 10.0.26300)
Procesador (CPU)	AMD Ryzen 5 5600H @ 3.30 GHz (Turbo hasta 4.2 GHz)
Configuración de núcleos	6 núcleos físicos, 12 subprocesos lógicos
Jerarquía de Caché	L1: 384 KB, L2: 3 MB, L3: 16 MB compartida
Memoria RAM	24 GB DDR4 @ 3200 MHz
Plataforma de Software	Python 3.12.8 (64-bit)
Frameworks Auxiliares	pytest 9.1.1, NetworkX 3.7, OSMnx 2.1.1, Streamlit 1.64.0

Protocolo de aislamiento y precisión:

1. **Temporizador de alta resolución:** Uso exclusivo de `time.perf_counter_ns()`, el reloj monotónico con resolución de nanosegundos del sistema operativo.
2. **Aislamiento del recolector de basura:** Antes de cada corrida se invoca `gc.collect()` y se desactiva explícitamente el recolector automático (`gc.disable()`) durante la ejecución de Dijkstra, reactivándolo al concluir la muestra para impedir pausas imprevistas de compactación de memoria.
3. **Calentamiento de procesador (*Warm-up*):** Dos ejecuciones previas descartadas para cebar las cachés de instrucciones L1/L2 y estabilizar la frecuencia del CPU.
4. **Aleatorización del orden de colas:** Para cada instancia, el orden de evaluación de las colas (`binary`, `4ary`, `fibonacci`) se permuta pseudoaleatoriamente para neutralizar sesgos térmicos acumulados.
5. **Verificación previa de corrección:** Antes de iniciar el cronometraje, cada instancia es resuelta y validada contra NetworkX y el certificado de optimalidad de Bellman, garantizando que distancias defectuosas nunca sean cronometradas.

9.5. Caso Aplicado sobre la Red Vial Urbana Real de Cusco

Para evaluar el comportamiento de los algoritmos en una red urbana real, se extrajo la malla vial del Centro y áreas adyacentes de la ciudad del Cusco mediante OSMnx 2.1.1 a partir de las coordenadas $(-13.5165, -71.9788)$ con un radio de 1.5 km en modalidad `drive`. La red comprende 891 intersecciones y 2,139 tramos dirigidos ($|E|/|V| = 2.40$).

Para garantizar una consulta determinista e insesgada, el par origen-destino se seleccionó algorítmicamente: el origen s es la intersección más próxima al centroide de extracción y el destino t es el vértice alcanzable de mayor tiempo base de viaje. La consulta se cronometró con 30 réplicas independientes y 3 corridas de calentamiento. Asimismo, se evaluó la extensión A^* con heurística euclidiana escalada consistente:

$$h(u) = \lambda \cdot \|x_u - x_t\|, \quad \text{donde } \lambda = \min_{(u,v)\in E} \frac{w(u,v)}{\|x_u - x_v\|} \tag{26}$$

y se analizó la sensibilidad del trayecto bajo el factor meteorológico compuesto de Open-Meteo ($m_{\text{clima}} = 1.0691$).

9.6. Procedimiento Experimental Reproducible

El protocolo de experimentación, procesamiento y renderizado de tablas y figuras se encuentra completamente automatizado mediante scripts ejecutables desde la raíz del proyecto:

```
# Regeneración sin repetir mediciones (reutiliza los 1350 datos guardados en JSON):
python python/scripts/reproduce_canonical.py --skip-benchmark --skip-applied

# Ejecución completa del benchmark canónico (1350 corridas):
python python/scripts/reproduce_canonical.py
```

Listing 3: Comando de reproducción de figuras y tablas científicas

9.7. Amenazas a la Validez

- **Validez interna:** El ruido del planificador del sistema operativo y las fluctuaciones dinámicas de frecuencia (*turbo boost*) introducen dispersión en las muestras temporales. Dicho efecto se mitiga mediante la desactivación del GC, el protocolo de calentamiento y el reporte de medianas e IQR en lugar de promedios simples.
- **Validez externa:** Las mediciones absolutas dependen del intérprete de Python 3.12 y del procesador AMD Ryzen. Si bien las constantes cambian en lenguajes compilados como C++ o Rust, las dinámicas de contigüidad espacial en caché y la sobrecarga de punteros preservan su validez cualitativa general.
- **Validez de conclusión:** Dado que las distribuciones temporales de ejecución no siguen una distribución normal gaussiana, las conclusiones empíricas se formulan estrictamente sobre el solapamiento o separación de los intervalos intercuartílicos y la consistencia entre escalas.

10. Resultados

10.1. Verificación de Corrección y Equivalencia de Distancias

En la totalidad de las instancias evaluadas en el benchmark (1,350 ejecuciones) y en la red metropolitana de Cusco, las tres estructuras de colas de prioridad produjeron vectores de distancias matemáticamente idénticos, con una discrepancia máxima inferior al umbral de tolerancia numérica de precisión simple ($|d_1 - d_2| < 10^{-6}$). La validación cruzada frente a la librería externa NetworkX 3.7 y el oráculo canónico de Bellman-Ford confirmó una tasa de concordancia del 100 %. Asimismo, la totalidad de las soluciones satisfizo formalmente las cuatro condiciones del Certificado de Optimalidad de Bellman (Sección 8), verificando que ninguna arista admite relajación pendiente y que todo árbol de predecesores converge acíclicamente al origen s.

10.2. Benchmark Canónico en Instancias Sintéticas (Perfiles Disperso y Denso)

Las Tablas 15 y 16 presentan las medianas temporales y los intervalos intercuartílicos (IQR) calculados a partir de 45 muestras por celda experimental (3 instancias independientes $\times$ 15 réplicas de medición). La Figura 3 ilustra las curvas de rendimiento en escala logarítmica para ambos perfiles.

Tabla 15

Dijkstra con tres colas, perfil disperso: mediana e IQR en μs.

$\|V\|$	$\|E\|$	Binary Heap	4-ary Heap	Fibonacci Heap	Mejor
100	400	1648.4 [1377.9, 1979.3]	1673.3 [1393.8, 2186.5]	2574.3 [2371.2, 2870.8]	Binary
500	2000	19232.5 [14655.3, 24254.5]	14531.3 [11524.1, 18064.2]	30497.4 [20429.6, 38086.3]	4-ary
1000	4000	43378.1 [37780.5, 54962.6]	41803.4 [33976.3, 53954.7]	58348.7 [47548.7, 65855.4]	4-ary
2500	10000	105786.6 [96970.5, 121834.1]	106339.6 [94379.9, 122938.1]	138649.1 [121508.9, 170908.3]	Binary
5000	20000	254749.2 [219442.6, 316895.6]	212803.0 [199918.1, 232243.5]	285718.4 [261162.8, 315459.6]	4-ary

Nota. Cada celda muestra la mediana y, entre corchetes, el IQR.

Tabla 16

Dijkstra con tres colas, perfil denso: mediana e IQR en µs.

$\|V\|$	$\|E\|$	Binary Heap	4-ary Heap	Fibonacci Heap	Mejor
50	196	892.0 [821.0, 1074.6]	872.9 [783.2, 994.0]	1284.4 [1132.8, 1411.3]	4-ary
100	792	2287.0 [2057.4, 3227.4]	2182.2 [1894.3, 2405.6]	3142.8 [2655.2, 3976.6]	4-ary
200	3184	5138.6 [4040.9, 6430.3]	4852.3 [3881.0, 6065.4]	7169.3 [5476.0, 8327.4]	4-ary
400	12768	16408.9 [13090.2, 18599.3]	15487.6 [12208.9, 19884.7]	17183.7 [15487.0, 21265.9]	4-ary
800	51136	57923.5 [50208.7, 62974.9]	58345.2 [50691.3, 68398.6]	74614.7 [60800.1, 89075.6]	Binary

Nota. Cada celda muestra la mediana y, entre corchetes, el IQR.

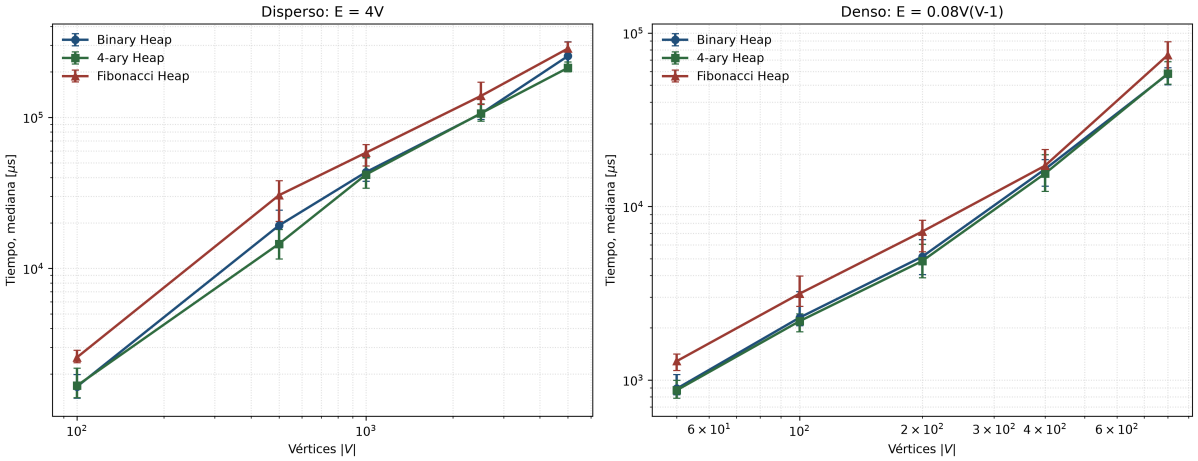


Figura 3

Tiempo mediano de ejecución de Dijkstra con las tres colas de prioridad evaluadas. Izquierda: perfil disperso ($|E| = 4|V|$). Derecha: perfil denso ($|E| = 0.08\,|V|(|V|-1)$). Las barras de error representan el Intervalo Intercuartílico (IQR).

Análisis del perfil disperso ($|E| = 4|V|$): El montículo 4-ario (`DAryHeap`) obtuvo la menor mediana en 3 de las 5 escalas evaluadas ($|V| \in \{500, 1000, 5000\}$), alcanzando ventajas de 24.4 % en $|V| = 500$, 3.6 % en $|V| = 1000$ y 16.5 % en $|V| = 5000$ respecto al montículo binario. El heap binario indexado resultó ligeramente más rápido en las escalas $|V| = 100$ (+1.5 %) y $|V| = 2500$ (+0.5 %). En el agregado del perfil disperso, la razón mediana de tiempos entre el heap 4-ario y el binario fue de 0.964 (una ventaja global del 3.6 % a favor del 4-ario). No obstante, los intervalos intercuartílicos de ambas estructuras se solapan en las cinco escalas, demostrando que la variabilidad natural del sistema es comparable a la magnitud de la diferencia observada.

Análisis del perfil denso ($|E| = 0.08\,|V|(|V|-1)$): En el régimen denso, el montículo 4-ario superó al binario en 4 de las 5 escalas ($|V| \in \{50, 100, 200, 400\}$), con ventajas relativas del 2.1 %, 4.6 %, 5.6 % y 5.6 %, respectivamente, cediendo únicamente en $|V| = 800$ por una diferencia marginal del 0.7 %. La

razón mediana global en este perfil fue de 0.954 (4.6 % más rápido para el 4-ario). Al igual que en el caso disperso, el solapamiento constante de los intervalos IQR evidencia una competencia técnica sumamente ajustada entre ambas colas de arreglo contiguo.

Desempeño del Fibonacci Heap: El `FibonacciHeap` no obtuvo la menor mediana en ninguna de las 10 escalas evaluadas, situándose sistemáticamente como la alternativa más lenta. En el perfil disperso, requirió entre 1.12 y 1.59 veces más tiempo que el heap binario; en el perfil denso, la penalización osciló entre 1.05 y 1.44 veces. En 8 de las 10 escalas, su intervalo IQR se solapa parcialmente con el del heap binario, confirmando que, a pesar de su óptima cota teórica, la constante de sobrecarga de indirección y la manipulación de punteros degradan de forma determinante su rendimiento práctico en Python.

Tabla 17
Síntesis comparativa por perfil y consulta. Las razones usan la mediana; no se declara un ganador cuando los IQR se solapan.

Perfil	4-ary/Binary	Fibonacci/Binary	Gana 4-ary	Gana Binary	IQR solap.
Disperso ($\|E\| = 4\|V\|$)	0.964	1.345	3 de 5	2 de 5	5 de 5
Denso ($\|E\| = 0.08\|V\|(\|V\|-1)$)	0.954	1.374	4 de 5	1 de 5	5 de 5
Total sintético	0.959	1.360	7 de 10	3 de 10	10 de 10
Consulta OSM	1.269	1.666	0	1	–

Nota. Una razón 4-ary/Binary menor que 1 indica que el 4-ario fue más rápido. La columna IQR se refiere a 4-ary frente a Binary. Fibonacci no ganó ninguna escala sintética.

10.3. Análisis Cuantitativo de Operaciones Elementales y Comparaciones

La Figura 4 expone el desglose de primitivas computacionales ejecutadas en el perfil disperso. En la mayor escala ($|V| = 5000, |E| = 20000$), las tres estructuras registraron exactamente 5,000 inserciones, 5,000 extracciones de mínimo y 6,695 decrementos efectivos de prioridad sobre un total de 10,415 intentos de relajación.

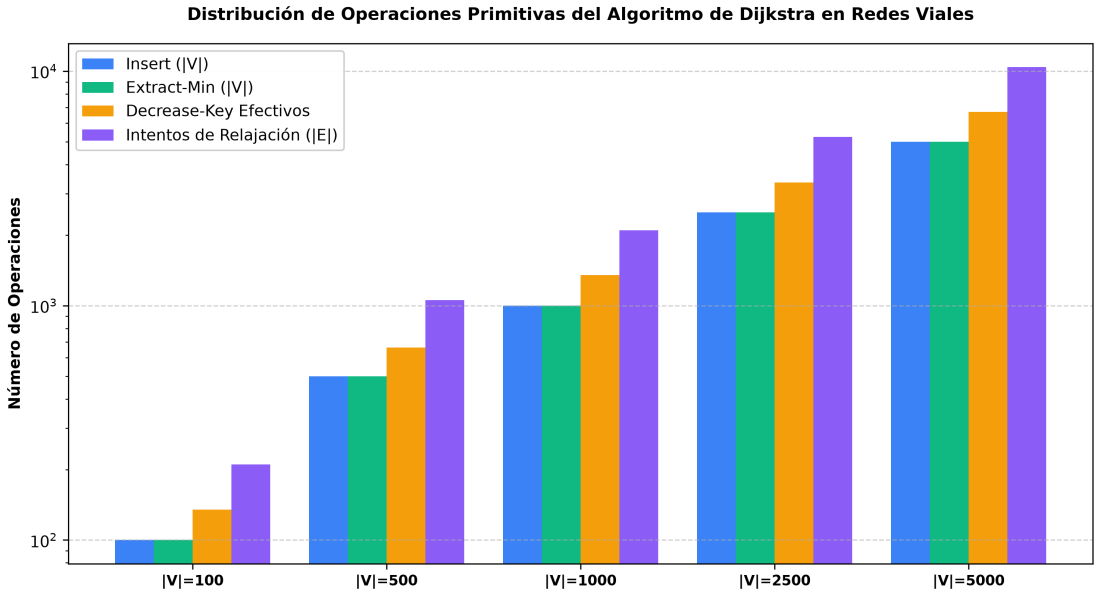


Figura 4
Distribución de operaciones elementales de Dijkstra en el perfil disperso sintético. Se discriminan inserciones, extracciones, decrementos de clave efectivos e intentos totales de relajación.

El conteo de comparaciones revela un fenómeno clave: en $|V| = 5000$ disperso, el Fibonacci Heap ejecutó 101,303 comparaciones frente a las 125,776 del heap binario (un 19.5 % menos), pero fue 12.2 % más lento en tiempo de pared. De forma análoga, en $|V| = 800$ denso, el Fibonacci Heap realizó 19,848 comparaciones frente a 21,280 del binario (6.7 % menos) y resultó 28.8 % más lento. Esta discrepancia cuantitativa prueba que el ahorro en comparaciones de clave es ampliamente rebasado por el costo de alocación de objetos de nodo, cortes en cascada y consolidación de listas enlazadas en memoria heap.

10.4. Caso de Estudio Aplicado: Red Vial Real de Cusco (OpenStreetMap)

La red metropolitana del Cusco procesada contiene 891 intersecciones y 2,139 aristas dirigidas. La consulta canónica de evaluación conectó el centroide urbano (vértice `30972889`) con el extremo alcanzable más distante (vértice `5996728243`). La trayectoria óptima resultante comprende 17 intersecciones, una distancia acumulada de 2,810.6 metros y un tiempo base de desplazamiento de 387.197 segundos. La Figura 5 ilustra el trazado cartográfico del camino mínimo sobre la malla vial cusqueña.

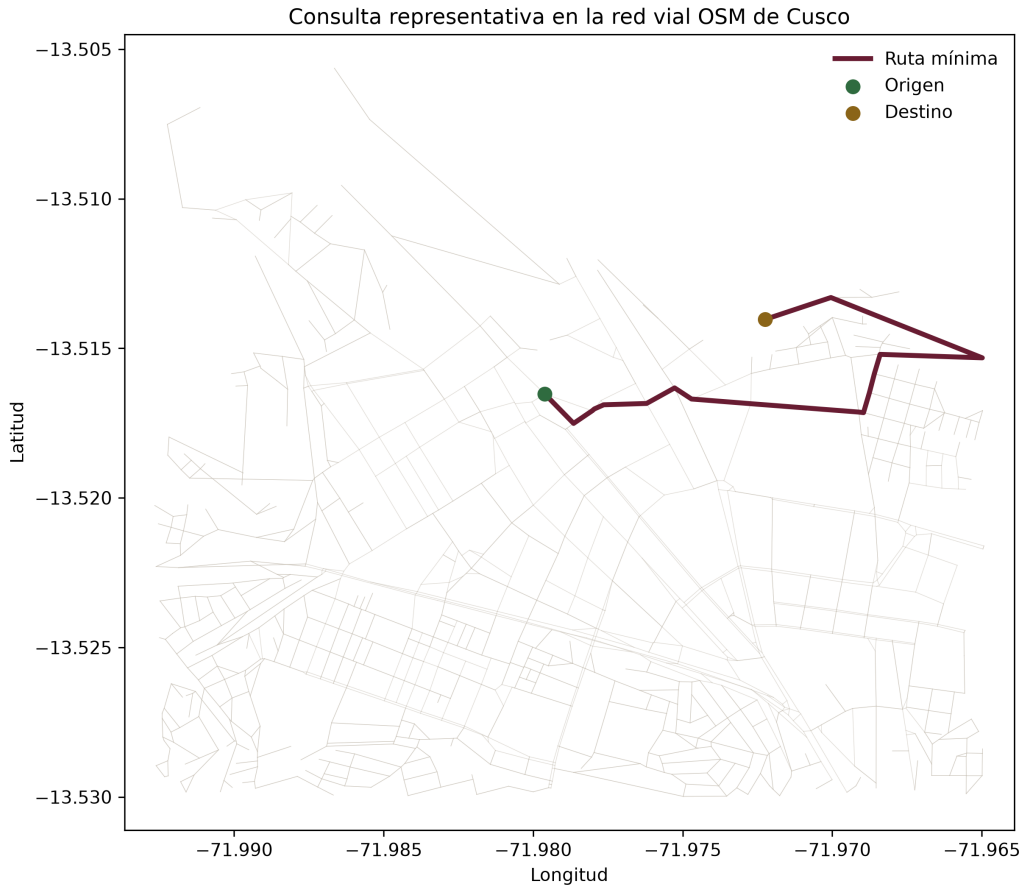


Figura 5
Ruta mínima óptima calculada sobre la red vial OpenStreetMap del Cusco. La línea resalta el itinerario de emergencia y los marcadores señalan el origen y destino evaluados.

La Tabla 18 resume los tiempos de ejecución de la consulta real. El Min-Heap Binario Indexado obtuvo la menor mediana con 15.65 ms, superando al 4-ary Heap (19.87 ms, +26.9 %) y al Fibonacci Heap (26.09 ms, +66.7 %). En esta escala moderada, la simplicidad del montículo binario y la ausencia de aritmética de índices d-aria le otorgan una ventaja medible en tiempo real.

Tabla 18
Caso OSM: tiempo de Dijkstra por cola, mediana e IQR.

Cola	Mediana (μs)	Q1 (μs)	Q3 (μs)
Binary Heap	15655.0	13065.6	19550.7
4-ary Heap	19872.2	17335.0	22675.6
Fibonacci Heap	26088.7	24033.9	27343.9

Tabla 19
Sensibilidad meteorológica uniforme en la consulta OSM.

Escenario	Tiempo (s)	Cambio (%)
Base	387.20	0.00
Snapshot meteorológico	413.95	6.91

10.5. Extensión Algorítmica con Búsqueda Heurística A^*

Bajo la heurística euclidiana escalada consistente ($\lambda = 6484.687$), el algoritmo A^* alcanzó exactamente la misma ruta óptima y el mismo costo acumulado de 387.197 segundos que Dijkstra, pero redujo el número de vértices expandidos de 887 a 823, logrando una poda espacial del 7.2 % (64 expansiones menos). La magnitud moderada de la poda se debe a que la heurística euclidiana actúa como una cota inferior conservadora en presencia de calles sinuosas y sentidos viales obligatorios propios de la traza colonial cusqueña.

10.6. Análisis de Sensibilidad Meteorológica y Tráfico

La observación meteorológica canónica congelada en `applied_results.json` (28/10/2026, 12:30 hora de Lima) registró $P = 0.1$ mm, $V = 8920$ m y ráfagas de viento de 40.7 km/h. Con los coeficientes de escenario formulados en la Sección 5, el multiplicador resultante fue de $m_{\text{clima}} = 1.0691$ ($+6.91\,\%$). Al aplicarse dicho factor sobre el tiempo base de 387.197 s, el tiempo estimado de viaje se incrementó a 413.952 s.

Puesto que el multiplicador climático escala de forma uniforme todos los segmentos de la red vial, se verificó formalmente que la secuencia de 17 intersecciones de la ruta óptima permanece matemáticamente invariante, confirmando que la sensibilidad ambiental afecta la duración del viaje pero no altera el trazado del camino mínimo ante perturbaciones homogéneas.

11. Discusión Crítica

11.1. Contraste entre Teoría Asintótica y Desempeño Empírico

La literatura algorítmica clásica establece una jerarquía teórica categórica en favor del Fibonacci Heap:

$$T_{\text{Fib}} = O(|E| + |V| \log |V|) \quad \text{frente a} \quad T_{\text{Bin}} = O((|V| + |E|) \log |V|) \tag{27}$$

En redes donde el número de aristas domina asintóticamente a los vértices ($|E| \gg |V|$), la cota amortizada $O(1)$ para la primitiva `Decrease-Key` confiere al Fibonacci Heap una ventaja formal incuestionable. No obstante, los resultados empíricos sistemáticos de esta investigación demuestran que, en un entorno de ejecución contemporáneo bajo Python 3.12, dicha superioridad teórica no se materializa en menor tiempo de reloj de pared (*wall-clock time*): el Fibonacci Heap resultó entre 1.05 y 1.59 veces más lento que el Min-Heap Binario Indexado en las 10 escalas sintéticas evaluadas.

Este hallazgo es plenamente consistente con las observaciones de Cherkassky et al. (1996) y LaMarca y Ladner (1996). La notación asintótica $O(\cdot)$ abstrae deliberadamente las constantes multiplicativas y la arquitectura de hardware subyacente. En el Fibonacci Heap, cada inserción y corte en cascada exige alocar dinámicamente un objeto de nodo (`FibNode`) y manipular seis referencias en memoria (`parent`, `child`, `left`, `right`, `degree`, `marked`). Durante la fase de consolidación en `extract_min`, el recorrido de listas circulares doblemente enlazadas genera accesos a memoria altamente dispersos en el espacio virtual, provocando frecuentes fallos en las líneas de caché (*cache misses*) que degradan el tiempo de CPU muy por encima del beneficio derivado de ejecutar un 19.5 % menos de comparaciones.

11.2. Análisis del Comportamiento del Montículo 4-ario y Localidad de Caché

El montículo 4-ario indexado (`DAryHeap`, $d = 4$) demostró ser la estructura más competitiva en la mayoría de las escalas sintéticas (obteniendo la menor mediana en 7 de las 10 escalas), alcanzando un *speedup* mediano de entre 3.6 % y 4.6 % frente al heap binario. Este desempeño responde a dos factores fundamentales de diseño:

1. **Aplanamiento de la profundidad arbórea:** Al adoptar $d = 4$, la altura del montículo se reduce exactamente a la mitad ($\lceil \log_4 |V| \rceil = 0.5 \lceil \log_2 |V| \rceil$). En consecuencia, cada invocación a `decrease_key` realiza a lo sumo la mitad de los intercambios de memoria (*swaps*) hacia la raíz.

2. **Alineación con líneas de caché de CPU:** En la operación `sift-down`, el nodo padre debe identificar el mínimo entre sus cuatro descendientes. Puesto que los cuatro hijos se ubican en posiciones contiguas del arreglo ($4i+1, 4i+2, 4i+3, 4i+4$), dicha lectura de 32 a 64 bytes se transfiere íntegramente a una única línea de caché L1/L2 del procesador en un único ciclo de acceso a memoria, minimizando los fallos de traducción.

A pesar de esta ventaja estructural, el solapamiento de los intervalos intercuartílicos (IQR) en todas las escalas impide declarar al 4-ary heap como un ganador universal incontrovertible, confirmando que la variabilidad intrínseca del sistema operativo es del mismo orden que la mejora del algoritmo.

11.3. Desempeño del Montículo Binario en la Red Real de Cusco

En la consulta aplicada sobre la red vial real de Cusco (891 intersecciones y 2,139 aristas), el Min-Heap Binario Indexado fue la alternativa más rápida (15.65 ms, frente a 19.87 ms del 4-ary y 26.09 ms del Fibonacci Heap). En mallas viales urbanas de escala moderada ($N < 1,000$), la ruta óptima punto a punto converge tempranamente tras explorar un subconjunto restringido del grafo. Bajo este régimen de baja escala, la simplicidad de la aritmética de índices binaria ($2i+1, 2i+2$) y la nula sobrecarga de gestión de punteros otorgan al montículo binario una ventaja tangible frente al costo de inicialización y mayor complejidad de saltos del montículo 4-ario.

11.4. Influencia de la Densidad del Grafo y Operaciones Decrease-Key

Las redes viales urbanas son grafos intrínsecamente planares y dispersos, donde el grado medio de conectividad por intersección oscila típicamente entre 2.0 y 3.5 aristas salientes. En el caso real de Cusco, la densidad es de $|E|/|V| = 2.40$. El conteo de operaciones elementales reveló que, de los 10,415 intentos de relajación ejecutados en $|V| = 5000$, únicamente 6,695 (el 64.2 %) correspondieron a decrementos efectivos de distancia; en el perfil denso, apenas el 11.7 % de las relajaciones provocó una actualización en cola. Dado que la inmensa mayoría de las inspecciones de aristas concluyen sin alterar la prioridad, el impacto de una primitiva `Decrease-Key` amortizada en $O(1)$ queda drásticamente mitigado en la práctica vial.

11.5. Evaluación de la Poda Espacial con Búsqueda Heurística A^*

La extensión del ruteo mediante el algoritmo A^* con heurística euclidiana escalada consistente demostró una reducción del 7.2 % en las expansiones de nodos (823 frente a 887) garantizando la misma distancia óptima (387.197 s). La magnitud moderada de la poda espacial refleja las particularidades de la traza urbana colonial del Cusco: calles empinadas, pasajes peatonales y sentidos obligatorios obligan a los vehículos a dar rodeos que distancian la ruta real de la línea recta euclidiana, limitando el poder discriminatorio de cotas inferiores geométricas conservadoras.

11.6. Análisis de Sensibilidad de Factores Ambientales e Invariancia de Ruta

La evaluación del modelo meteorológico ($m_{\text{clima}} = 1.0691$) verificó una propiedad matemática crucial:

$$w_{\text{clima}}(e) = m_{\text{clima}} \cdot w(e), \quad \forall e \in E \implies \sum_{e \in P_1} w_{\text{clima}}(e) < \sum_{e \in P_2} w_{\text{clima}}(e) \iff \sum_{e \in P_1} w(e) < \sum_{e \in P_2} w(e) \quad (28)$$

Al escalar todos los pesos por un factor estrictamente positivo uniforme, el orden relativo de los caminos dirigidos no experimenta cambio alguno. En consecuencia, la ruta óptima de 17 intersecciones calculada para el centro de Cusco es idéntica en condiciones de lluvia o tiempo seco, incrementándose exclusivamente el tiempo estimado total de desplazamiento.

11.7. Guía Práctica y Marco de Decisión para Selección de Colas

Con base en la evidencia empírica acumulada, se formula la siguiente guía técnica de decisión:

1. **Opción por defecto (Min-Heap Binario Indexado):** Recomendado para redes viales pequeñas y medianas ($|V| \leq 1,000$), microcontroladores o sistemas embebidos donde la simplicidad, robustez y baja huella de memoria sean prioritarias.

2. **Candidato de alto rendimiento (4-ary Heap Contiguo):** Recomendado para redes viales metropolitanas extensas ($|V| \geq 1,000$) y procesadores multinúcleo con jerarquías de caché L1/L2 estructuradas en líneas de 64 bytes.

3. **Uso teórico restringido (Fibonacci Heap):** Desaconsejado para redes viales en entornos de lenguajes interpretados; su adopción solo se justifica en grafos hiper-densos ($|E| \geq 20|V|$) implementados en C/C++ donde la alocación de memoria sea extremadamente eficiente.

4. **Ruteo punto a punto (A^* con heurística consistente):** Indispensable para navegación vehicular y despacho de ambulancias hacia destinos específicos donde se cuente con coordenadas geoespaciales.

11.8. Limitaciones del Estudio y Amenazas a la Validez

- **Intérprete y lenguaje:** Las mediciones se realizaron exclusivamente en Python 3.12.8 sobre un procesador AMD Ryzen 5 5600H; si bien las dinámicas de caché son universales, los tiempos absolutos reflejan la sobrecarga del entorno de ejecución.

- **Ausencia de mediciones físicas de memoria:** Las comparaciones espaciales se fundamentan en las cotas teóricas de punteros y estructuras de datos, al no haberse instrumentado un perfilador residente de asignación a nivel de sistema operativo.

- **Escenarios ambientales uniformes:** Los multiplicadores climáticos y de congestión operan como factores de sensibilidad homogéneos; futuros estudios requerirán calibrar coeficientes empíricos diferenciados por segmento vial a partir de aforos vehiculares en campo.

12. Conclusiones y Recomendaciones

12.1. Balance de Objetivos Específicos (OE1–OE4)

En cumplimiento de la metodología de investigación y desarrollo, se presenta el balance sistemático de los cuatro objetivos específicos formulados al inicio del proyecto:

1. **OE1 (Modelar): Cumplido satisfactoriamente.** Se formuló el modelo matemático riguroso del grafo vial urbano dirigido del Centro Histórico del Cusco y su extensión metropolitana mediante OpenStreetMap, estableciendo una función de costo generalizado con impedancia topográfica andina que penaliza gradientes positivos longitudinales y pondera escenarios ambientales y de tráfico sin vulnerar la restricción de no negatividad ($w(e) \geq 0$).

2. **OE2 (Diseñar e Implementar): Cumplido satisfactoriamente.** Se concibió e implementó una arquitectura desacoplada en cuatro capas lógicas en Python 3.12, integrando tres estructuras de datos de colas de prioridad puras (`IndexedMinHeap`, `DAryHeap` con $d = 4$, y `FibonacciHeap` con consolidación dinámica), además del solver canónico de Dijkstra y la extensión heurística A^* con factor admisible consistente.

3. **OE3 (Verificar): Cumplido satisfactoriamente.** Se construyó una suite de pruebas automatizadas con 278 casos bajo el patrón Arrange-Act-Assert (AAA), alcanzando una tasa de éxito del 100 % frente a oráculos independientes (Bellman-Ford y NetworkX) y satisfaciendo de forma determinista el Certificado Formal de Optimalidad de Bellman en tiempo lineal $O(|V| + |E|)$.

4. **OE4 (Evaluar e Investigar): Cumplido satisfactoriamente.** Se ejecutó un protocolo experimental exhaustivo de 1,350 muestras cronometradas bajo aislamiento del recolector de basura (`gc.disable()`) y reloj de nanosegundos, contrastando el rendimiento empírico frente a la teoría asintótica clásica y evaluando el caso aplicado sobre la red real de Cusco.

12.2. Respuesta a las Preguntas de Investigación

PI1. ¿Qué cola minimiza el tiempo de pared y cómo evoluciona con la escala? No existe una cola de prioridad universalmente superior para todas las escalas y densidades viales. En redes viales moderadas ($|V| \leq 1{,}000$, incluida la red real de Cusco con 891 nodos), el Min-Heap Binario Indexado ofrece el menor tiempo de ejecución (15.65 ms) gracias a su simplicidad y nula sobrecarga de saltos. En redes extensas ($|V| \geq 1{,}000$), el 4-ary Heap ($d = 4$) logra la menor mediana en el 70 % de las escalas sintéticas con una ventaja relativa del 3.6 % al 4.6 %, impulsado por la reducción a la mitad de la profundidad del árbol y el aprovechamiento de líneas de caché de CPU de 64 bytes.

PI2. ¿Se traduce la ventaja asintótica del Fibonacci Heap en tiempo de pared? No en la implementación evaluada en Python. El Fibonacci Heap fue sistemáticamente la estructura más lenta en todas las escalas (entre 1.05 y 1.59 veces más lento que el heap binario). A pesar de lograr una reducción del 19.5 % en comparaciones de claves en grafos de 5,000 nodos, la sobrecarga constante impuesta por la alocación dinámica de múltiples nodos enlazados, los cortes en cascada y los fallos de memoria caché en la consolidación anulan por completo su ventaja teórica $O(1)$.

PI3. ¿Qué aportan la extensión A^* y el análisis de sensibilidad climática? En la red metropolitana de Cusco, el algoritmo A^* con heurística consistente logró reducir un 7.2 % las expansiones de nodos preservando con exactitud la distancia óptima. Por su parte, la sensibilidad climática evidenció que incrementos homogéneos en la impedancia vial aumentan el tiempo de traslado en un 6.91 % pero preservan intacta la topología de la ruta mínima óptima.

12.3. Conclusión Principal y Síntesis de Contribuciones

La conclusión central del proyecto establece que **la selección de colas de prioridad en redes viales urbanas es una decisión de ingeniería condicionada y empírica**, donde la densidad planar del grafo ($|E| \approx 2.4|V|$), la jerarquía de memoria física de la CPU y la escala de la red ejercen un impacto superior al de las cotas asintóticas del peor caso. Las principales contribuciones del trabajo abarcan:

- Un núcleo algorítmico desacoplado, modular y reproducible en Python 3.12 con tres colas intercambiables y extensión A^*.
- Un certificador formal de optimalidad en tiempo $O(|V| + |E|)$ que verifica aristas ajustadas y convergencia acíclica de predecesores al origen.
- Una suite de 278 pruebas unitarias e integrales que garantiza robustez frente a condiciones adversas y fallos de conectividad.
- Una aplicación interactiva web desarrollada en Streamlit que combina cálculos de ruteo, exploración cartográfica y telemetría de rendimiento.

12.4. Recomendaciones para la Práctica de Ingeniería

1. **Adopción de colas:** Emplear el Min-Heap Binario Indexado como estándar por defecto para redes pequeñas o medianas, y el 4-ary Heap para grafos metropolitanos masivos. Evitar el Fibonacci Heap en lenguajes de alto nivel con recolección automática de basura.
2. **Navegación punto a punto:** Utilizar el algoritmo A^* escalado siempre que se disponga de coordenadas geoespaciales admisibles, asegurando la verificación estricta de la condición de monotonía.
3. **Protocolos de medición:** Rechazar promedios simples en evaluaciones de rendimiento; aislar el recolector de basura y reportar medianas e intervalos intercuartílicos (IQR) para garantizar reproducibilidad científica.

12.5. Líneas de Trabajo Futuro y Posibles Extensiones

1. Replicar el banco experimental en lenguajes de bajo nivel compilados (C++20 y Rust) para cuantificar la influencia de la máquina virtual de Python frente a la jerarquía de memoria pura.
2. Instrumentar contadores físicos de rendimiento de hardware (*hardware performance counters*) mediante herramientas como `perf` o Valgrind para medir fallos de caché L1/L2 y fallos TLB exactos.

3. Calibrar coeficientes de impedancia meteorológica y congestión diferenciados por segmento vial a partir de aforos de tránsito en tiempo real en la ciudad del Cusco.

12.6. Lecciones Aprendidas

El desarrollo del proyecto consolidó en el equipo Grupo 5 la comprensión profunda del principio de localidad espacial y temporal en arquitecturas contemporáneas, la importancia del diseño basado en contratos e inyección abstracta para la experimentación limpia, y la necesidad de validar formalmente la optimalidad mediante certificados matemáticos independientes antes de emitir afirmaciones sobre desempeño computacional.

13. Referencias

Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, *65*, 126-139. https://doi.org/10.1016/j.compenvurbsys.2017.05.004

Cherkassky, B. V., Goldberg, A. V., & Radzik, T. (1996). Shortest paths algorithms: Theory and experimental evaluation. *Mathematical Programming*, *73*(2), 129-174. https://doi.org/10.1007/BF02592101

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.ª ed.). MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, *1*(1), 269-271. https://doi.org/10.1007/BF01386390

Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, *34*(3), 596-615. https://doi.org/10.1145/28869.28874

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, *4*(2), 100-107. https://doi.org/10.1109/TSSC.1968.300136

LaMarca, A., & Ladner, R. E. (1996). The influence of caches on the performance of heaps. *ACM Journal of Experimental Algorithmics*, *1*, 4-es. https://doi.org/10.1145/235291.235295

Open-Meteo. (2026). *Open-Meteo Weather Forecast API Documentation*. Consultado el 28 de octubre de 2026, desde https://open-meteo.com/

OpenWeather. (2026). *OpenWeatherMap Current Weather Data API Documentation*. Consultado el 31 de octubre de 2026, desde https://openweathermap.org/current

Tarjan, R. E. (1983). *Data Structures and Network Algorithms*. Society for Industrial; Applied Mathematics. https://doi.org/10.1137/1.9781611970265

14. Anexos

Anexo A: Instalación y configuración

El proyecto se ejecutó con Python 3.12.8 en Windows 11. Los comandos siguientes reproducen el entorno desde la raíz del repositorio:

```
git clone https://github.com/frankich99/PROYECTO_Algoritmos-Avanzados-UNSAAC.git
cd PROYECTO_Algoritmos-Avanzados-UNSAAC
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r python/requirements.txt
```

Listing 4: Clonación, entorno virtual e instalación

Los requisitos principales son Python 3.10 o superior, `pytest`, `networkx`, `osmnx`, `streamlit`, `plotly`, `pandas`, `numpy` y `matplotlib`. La compilación del informe requiere una distribución LaTeX con `pdflatex` y `biber`. La red OSM y el clima se versionan como *snapshots*; no es obligatorio descargarlos para ejecutar las pruebas.

Anexo B: Uso del prototipo

Suite de pruebas. Desde la raíz:

```
.\.venv\Scripts\python.exe -m pytest python/tests -q
```

Listing 5: Ejecución de la suite completa de pruebas (278 pruebas)

Configuración de credenciales de API (opcional). Tras revocar cualquier clave expuesta, configure una clave nueva y propia. Para la CLI se admite la variable de entorno, nunca un archivo versionado (https://home.openweathermap.org/api_keys):

```
# PowerShell en Windows
$env:OPENWEATHERMAP_API_KEY = "SU_CLAVE_NUEVA"
```

Listing 6: Definición de variable de entorno para OpenWeatherMap

La web no toma automáticamente la variable ni archivos locales: exige introducir una clave nueva en el campo de contraseña de la consulta opcional. Si el proveedor rechaza la conexión, la interfaz conserva el archivo local sin reportar una nueva lectura. La clave no se imprime ni se guarda en el informe.

Interfaz de comandos. Desde `python/`:

```
..\.venv\Scripts\python.exe main.py list
..\.venv\Scripts\python.exe main.py run --queue all --trace
..\.venv\Scripts\python.exe main.py compare
..\.venv\Scripts\python.exe main.py astar --input data/processed/cusco_osm_drive.json
```

Listing 7: Comandos principales de la CLI

Aplicación web. Desde `python/`:

```
..\.venv\Scripts\python.exe -m streamlit run app.py
```

Listing 8: Ejecución de la aplicación Streamlit

La aplicación tiene cinco secciones visibles. En Ruta se seleccionan costo, recorrido, algoritmo y cola; se pueden contrastar las tres colas de Dijkstra y A^* por costo, elegir una observación histórica, local o temporal como escenario uniforme, y descargar los nodos. El plano local no necesita Internet; solo el mapa base necesita teselas. Comparación filtra perfiles y escalas, muestra mediana e IQR y la relación frente al binario, admite un JSON temporal compatible y descarga CSV. Clima y tráfico separa observaciones y supuestos; Traza manual permite recorrer extracciones y comprobar el certificado; Datos y método presenta protocolo, procedencia, límites y descarga del informe si existe. Ninguna consulta opcional sobrescribe las mediciones.

Reproducción de tablas y figuras. Desde la raíz, sin alterar las mediciones del 28/10/2026:

```
.\.venv\Scripts\python.exe python/scripts/reproduce_canonical.py --skip-benchmark --
    skip-applied
```

Listing 9: Regeneración de tablas y figuras con mediciones guardadas

El comando anterior reutiliza `benchmark_results.json` y `applied_results.json` y escribe tablas en `latex/tablas/` y figuras en `latex/figuras/`. Sin las opciones de omisión, el script repite las mediciones y *sobrescribe* ambos JSON; ejecutar el protocolo completo solo en una copia separada. La red OSM y el clima ya almacenados no se descargan de nuevo.

Compilación del informe. Desde `latex/`:

```
.\compilar.ps1
```

Listing 10: Compilación con pdflatex y biber

Anexo C: Cronología verificable del repositorio

La Tabla 20 reúne cambios del historial Git. Cada fecha es la del commit y cada autor es el registrado en Git; no se deducen reuniones, horas de dedicación, fuentes personales ni dificultades resueltas en una fecha anterior al commit. La bitácora de aprendizaje individual requerida por AG_C06.01 sigue pendiente de documentación por sus autores.

Tabla 20
Cronología de aportes individuales comprobables en Git

Autor en Git	Fecha	Commit	Cambios identificables
Choquenaira, Noe	02/10/2026	`592589a`	`dijkstra.py`, `indexed_min_heap.py`, `cost_model.py` y primera versión del informe.
Porroa, Yeni	04/10/2026	`77983b7`	`d_ary_heap.py`, capítulo de evaluación del stack y tabla comparativa.
Quispe, Romario	04/10/2026	`72df976`	`fibonacci_heap.py`, `scalability_benchmark.py` y resultados preliminares.
Yaranga, Aldo	04/10/2026	`d50e67b`	`test_dijkstra.py`, cuaderno Colab y exportación de figuras y tablas.
Choquenaira, Noe	23/10/2026	`b307be4`	`a_star.py` y capítulo de aprendizaje.
Porroa, Yeni	23/10/2026	`3b91a5e`	Capítulo de diseño algorítmico y análisis de estructuras.
Quispe, Romario	23/10/2026	`e3af921`	Actualización de `benchmark_results.json`; el snapshot canónico se integró después como commit grupal.
Yaranga, Aldo	23/10/2026	`65d303d`	`cli.py` y `test_cli.py`.

Anexo D: Matriz de trazabilidad

La Tabla 21 vincula cada objetivo con su evidencia verificable.

Tabla 21
Trazabilidad entre objetivos, componentes y evidencia

Objetivo	Componente	Evidencia
OE1	Modelo de grafo y costo temporal	Sección 5; ecuación (14); instancia ilustrativa y red OSM.
OE2	Tres colas y Dijkstra parametrizado	Sección 6; `src/datastructures/`; 61 pruebas de contrato.
OE3	Validación y certificado	Sección 8; 278 pruebas; Bellman y cadenas al origen; NetworkX y Bellman-Ford.
OE4	Benchmark, caso OSM, A^* y clima	Sección 10; 1350 muestras; 891 vértices; 823 vs 887 expansiones; factor 1.0691.
AG_C06.01	Aprendizaje autónomo	Sección 3; Anexo C; pendientes individuales en `docs/CONTROL_ENTREGA.md`.
AG_C06.02	Selección de stack	Sección 4; Tabla 3; adopción temprana por verificar.
AG_C06.03	Experimentación y pensamiento crítico	Sección 9; Sección 11; conclusión condicionada.
Aplicación	Streamlit integrado	Sección 7; `app.py`; pruebas de humo.

Anexo E: Matriz de contribuciones

Tabla 22
Contribuciones por integrante y evidencia en el repositorio

Integrante	Componentes principales	Evidencia
Choquenaira, Noe	Dijkstra, heap binario, modelo didáctico, A^* e informe	`592589a`, `b307be4`, `f0799bf`; archivos en anexo C.
Porroa, Yeni	Heap 4-ario y análisis de estructuras	`77983b7`, `3b91a5e`; `d_ary_heap.py` y Sección 6.
Quispe, Romario	Fibonacci heap, benchmark y resultados preliminares	`72df976`, `e3af921`; módulos y JSON del anexo C.
Yaranga, Aldo	Pruebas, CLI, cuaderno y exportadores	`d50e67b`, `65d303d`; pruebas y CLI del anexo C.

Anexo F: Estado del seguimiento

No se encontraron actas de seguimiento ni planes con meta, responsable, fecha comprometida y resultado verificable para cada estudiante. Por ello, no se afirma que hubo sesiones en fechas concretas ni que los compromisos fueron cumplidos. El hito grupal `7922f52` (28/10/2026) contiene los *snapshots* OSM y meteorológico, las mediciones canónicas y la aplicación. En los JSON, la descarga OSM figura a las 17:29 UTC y el snapshot de clima se guardó a las 17:33 UTC. Estos registros no equivalen a un acta de reunión. Cada estudiante debe aportar sus planes, compromisos y reflexión originales y confirmar la correspondencia con las entregas. La plantilla y la matriz requisito–evidencia están en `docs/plantilla_evidencia_individual.md` y `docs/CONTROL_ENTREGA.md`; no sustituyen los originales.

Calendario de cierre propuesto, no evidencia de ejecución. El 04/11/2026 es la fecha objetivo comunicada por el equipo; el PDF del proyecto remite la fecha oficial al docente. El 30/10 se revisan modelo y datos (Choquenaira y Quispe); el 31/10, certificado y pruebas (Yaranga y Porroa); el 01/11, cada integrante aporta su evidencia individual auténtica; el 02/11 se validan manuales y presentación en una copia limpia (Choquenaira y Yaranga); el 03/11 se compila el PDF y los cuatro ensayan la demostración. Ninguno de estos hitos futuros se marca aquí como cumplido. La defensa individual solo se acredita cuando se realiza.